Proyecto SinOdio/SansHaine:

Clasificador automático de discurso de odio en español

(con fase prevista de extensión al francés)

Alumna:

Ariane Heinz Vallribera

Fecha:

Marzo-Abril 2026

Bootcamp de KeepCoding:

Big Data, Inteligencia Artificial, Machine Learning - XVI Edición, KeepCoding

Tutora:

Sandra Navarro

Memoria del Proyecto: SinOdio/SansHaine

Memoria del Proyecto: SinOdio/SansHaine..... 2

Resumen..... 4
Abstract..... 4

A. Organización general del proyecto..... 5
1. Descripción general..... 5
2. Gestión del proyecto..... 5
3. Datasets..... 7
4. Configuración del repositorio GitHub..... 8

B. Arquitectura e infraestructura..... 9
1. Configuración del proyecto en GCP..... 9
2. Control de costes..... 9
3. Cloud Storage..... 9

C. Configuración del repositorio y reproducibilidad..... 11
1. Repositorio GitHub..... 11
2. Reproducibilidad..... 12
3. utils.py..... 13

D. Infraestructura de datos..... 14
1. Adquisición de datos..... 14
2. Gestión de credenciales y seguridad..... 14

E. Análisis Exploratorio de Datos (EDA)..... 15
1. Estructura y tipos de datos..... 15
2. Distribución de clases..... 15
3. Distribución de longitud de textos..... 16
4. Valores nulos, duplicados y textos vacíos..... 18
5. Análisis de menciones (“@USER”)..... 19
6. Análisis de vocabulario por clase..... 19
7. Conclusiones del EDA y decisiones para el preprocesamiento..... 22

F. Preprocesamiento y ETL..... 23
1. Carga de datos desde GCP..... 23
2. Estandarización de columnas y tipos de datos..... 23
3. Eliminación de duplicados, etiquetas conflictivas y textos muy cortos..... 23
4. Limpieza de texto..... 24
5. Fusión del corpus..... 25
6. División en conjuntos de entrenamiento, validación y test..... 25
7. Almacenamiento y registro de métricas usando PySpark..... 25

G. Baseline de Machine Learning Tradicional....26
1. Extracción de características: TF-IDF....26
2. Análisis de n-gramas por clase....27
3. Modelos entrenados....28
4. Optimización de hiperparámetros con Optuna....29
5. Evaluación de los modelos ajustados....29
6. Selección y almacenaje del modelo baseline....30

H. Fine-tuning de XLM-RoBERTa....31
1. Infraestructura: VM en GCP con GPU....31
Configuración del entorno....31
Limitaciones de permisos de la VM....32
2. Tokenización....32
3. Arquitectura del modelo....33
4. Búsqueda de hiperparámetros con Optuna....33
5. Fine-tuning completo....34
6. Evaluación y comparación con el baseline....35
7. Análisis cualitativo del modelo....36
8. Almacenamiento....36

I. Despliegue del modelo: FastAPI y Cloud Run....37
1. Infraestructura y enfoque....37
2. Dependencias del contenedor....37
3. Incidencias durante el despliegue....37
4. Interacción con la API....38
5. Análisis cualitativo y limitaciones identificadas durante el despliegue....41

J. Visualización de datos y resultados con dashboards de PowerBI....43
1. Dashboard de motivación del proyecto....43
a) Objetivo....43
b) Fuente de datos....43
c) Visualizaciones....44
d) Vías de mejora....45
2. Dashboard de resultados del modelo....45
a) Conexión a BigQuery....45
b) Modelado de datos con DAX....46
c) Dashboard de Resultados....47

K. Conclusiones, lecciones aprendidas y trabajo futuro....50
1. Validación de las suposiciones iniciales....50
2. Valoración de las métricas seleccionadas....51
3. Valoración de la arquitectura elegida....51
4. Lecciones aprendidas....52
5. Trabajo futuro....53

Resumen

SinOdio/SansHaine es un clasificador automático de discurso de odio en español, con una extensión al francés prevista en la próxima fase, desarrollado como proyecto final del *KeepCoding Bootcamp | Big Data, Inteligencia Artificial & Machine Learning | Edición XVI*. El sistema se basa en el modelo transformer multilingüe XLM-RoBERTa, con fine-tuning sobre un corpus de ~57.000 ejemplos construído a partir de tres datasets públicos: el Spanish Hate Speech Superset (tweets), DETOXIS (comentarios de noticias sobre inmigración) y HAHA (humor ofensivo). La variedad intencional del corpus permite al modelo aprender representaciones robustas del discurso de odio más allá de un único registro lingüístico o grupo-objetivo.

El sistema se evalúa mediante F1 macro como métrica principal, adecuada al desequilibrio de clases del corpus (69% no odio / 31% odio) y al contexto de aplicación. XLM-RoBERTa alcanza un F1 macro de 0,810 en el conjunto de test, superando en 6,5 puntos porcentuales al baseline de Regresión Logística con TF-IDF (F1 = 0,745). El modelo fine-tuned se despliega como API REST en Google Cloud Run, con endpoints para clasificación individual y por lotes, y los resultados se almacenan en BigQuery para su visualización en dashboards interactivos de PowerBI.

El proyecto está diseñado como un sistema incremental de cuatro fases: la Fase 1, documentada en esta memoria, constituye un entregable completo y válido, mientras que las fases posteriores extienden el sistema al francés, a la clasificación multiclase por grupo-objetivo, y a la recuperación de contranarraciones validadas por ONGs.

Palabras-clave: discurso de odio, NLP, XLM-RoBERTa, clasificación binaria, español, redes sociales, transformers, FastAPI, Google Cloud Run, BigQuery.

Abstract

SinOdio/SansHaine is an automatic hate speech classifier for Spanish, with an extension to French in the next phase, developed as the final project of the *KeepCoding Big Data, Artificial Intelligence & Machine Learning Bootcamp, 16th Edition*. The system is based on XLM-RoBERTa, a multilingual transformer model, and is fine-tuned on a corpus of ~57,000 examples combining three public datasets: the Spanish Hate Speech Superset (tweets), DETOXIS (news comments on immigration) and HAHA (offensive humour). The intentional variety of the corpus encourages the model to learn robust representations of hate speech beyond a single linguistic register or target group.

The system is evaluated using macro F1 as the primary metric, appropriate for the class imbalance of the corpus (69% non-hate / 31% hate) and the intended application context. XLM-RoBERTa achieves a macro F1 of 0.810 on the test set, outperforming the Logistic Regression with TF-IDF baseline (F1 = 0.745) by 6.5 percentage points. The fine-tuned model is deployed as a REST API on Google Cloud Run, with endpoints for single and batch classification, and results are stored in BigQuery for visualisation in interactive PowerBI dashboards.

The project is designed as an incremental four-phase system: Phase 1 —documented in this report— constitutes a complete and self-contained deliverable, while subsequent phases extend the system to French, multi-class classification by target group, and retrieval of NGO-validated counter-narratives.

Keywords: hate speech, NLP, XLM-RoBERTa, binary classification, Spanish, social media, transformers, FastAPI, Google Cloud Run, BigQuery.

A. Organización general del proyecto

1. Descripción general

SinOdio/SansHaine es un clasificador automático de discurso de odio en español, con arquitectura preparada para su extensión al francés en la siguiente fase, basado en el modelo multilingüe preentrenado XLM-RoBERTa. El proyecto está orientado a servir como herramienta de apoyo para ONGs y organizaciones de derechos humanos, cubriendo grupos vulnerables como mujeres, inmigrantes, minorías regionales y comunidades LGTB. El sistema detecta discurso de odio en textos extraídos de redes sociales y comentarios de noticias, y en fases avanzadas recupera contranarraciones efectivas previamente validadas por ONGs.

El proyecto se estructura en cuatro fases incrementales, de manera que la Fase 1 presentada en esta memoria constituye por sí sola un entregable completo y válido, y cada fase posterior extiende las capacidades del sistema:

- **Fase 1**: **Clasificador binario en español (núcleo del proyecto)**
- **Fase 2**: Extensión a francés (rama A)
- **Fase 3**: Clasificador multiclase en español (rama B)
- **Fase 4**: Recuperación de contranarraciones (converge las ramas A y B)

Adicionalmente, el proyecto incluye un conjunto de entregables de comunicación: dos dashboards interactivos en PowerBI, un vídeo de presentación final, un pitch de plan de negocio, y la documentación técnica completa en GitHub <https://github.com/AriadnaHV/Fighting-Hate-Speech>.

2. Gestión del proyecto

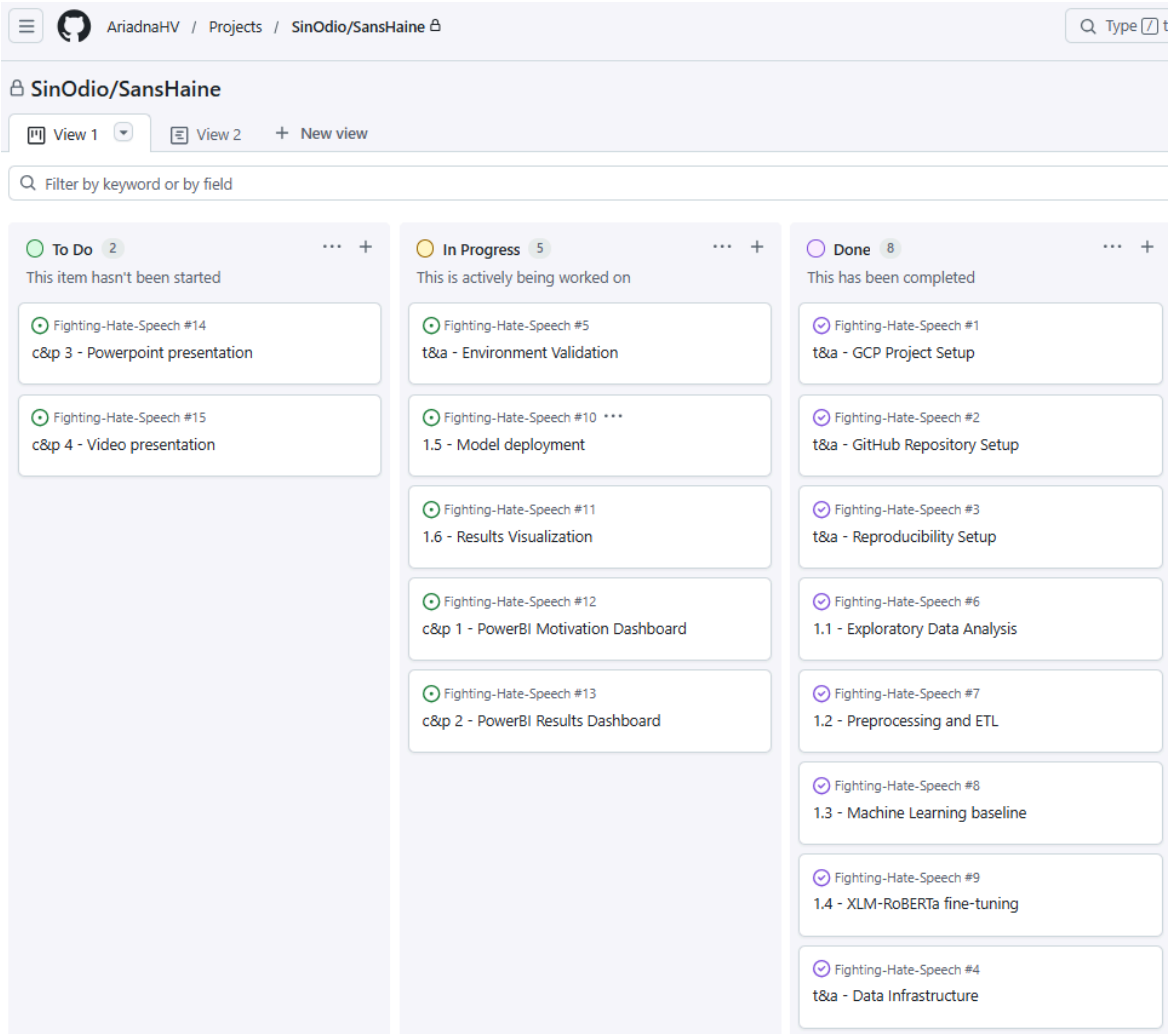
La metodología de desarrollo es **SCRUM**, con *milestones* y *sprints* de tareas. El seguimiento se realiza mediante **GitHub Projects**, con un tablero de tres columnas: *To Do*, *In Progress* y *Done*. Se descartó Trello en favor de GitHub Projects por su integración nativa con el repositorio de código, los *issues* y los *commits*, lo que permite una trazabilidad completa del trabajo realizado, y también porque se trata de la herramienta usada en la Masterclass de trabajo en equipo del bootcamp.

El proyecto se organiza en seis milestones:

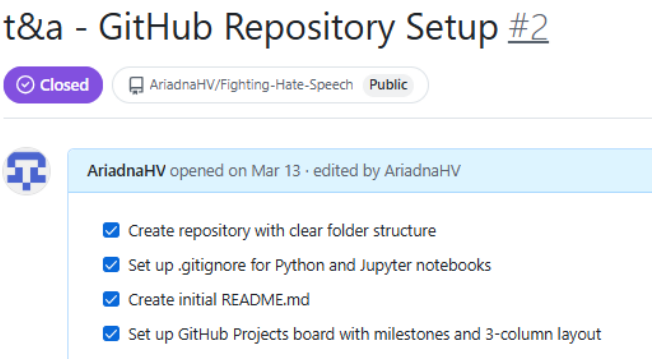
Milestone	Descripción
Tools & Architecture	Configuración de infraestructura y herramientas
Phase 1	Clasificador binario en español
Phase 2	Extensión a francés
Phase 3	Clasificador multiclase
Phase 4	Recuperación de contranarraciones
Communication	Dashboards, vídeo, pitch y documentación

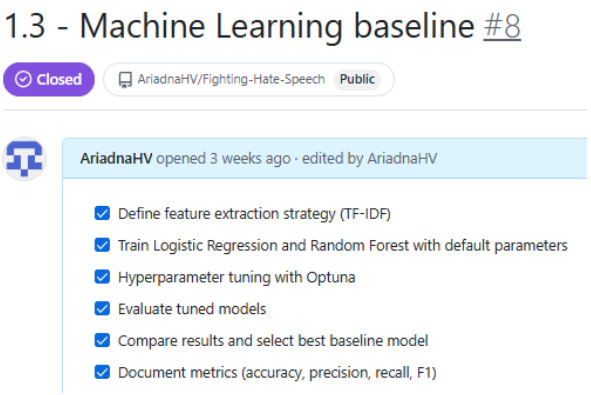
Cada *milestone* contiene *issues* con listas de tareas detalladas. Las *issues* están etiquetadas con labels por fase para facilitar el filtrado y seguimiento del progreso. Como se explicó en la descripción general, la Fase 1 del proyecto (conjuntamente con los milestones de *Tools & Architecture* y *Communication*) constituye por sí sola un proyecto completo y válido. No obstante, se intentará completar las fases restantes –si no dentro del plazo del bootcamp, en las semanas posteriores– dado el interés personal de la autora en la temática y las posibilidades de impacto social del proyecto para ONGs y organizaciones de derechos humanos.

La siguiente captura de pantalla muestra el estado de las tareas en un momento concreto:

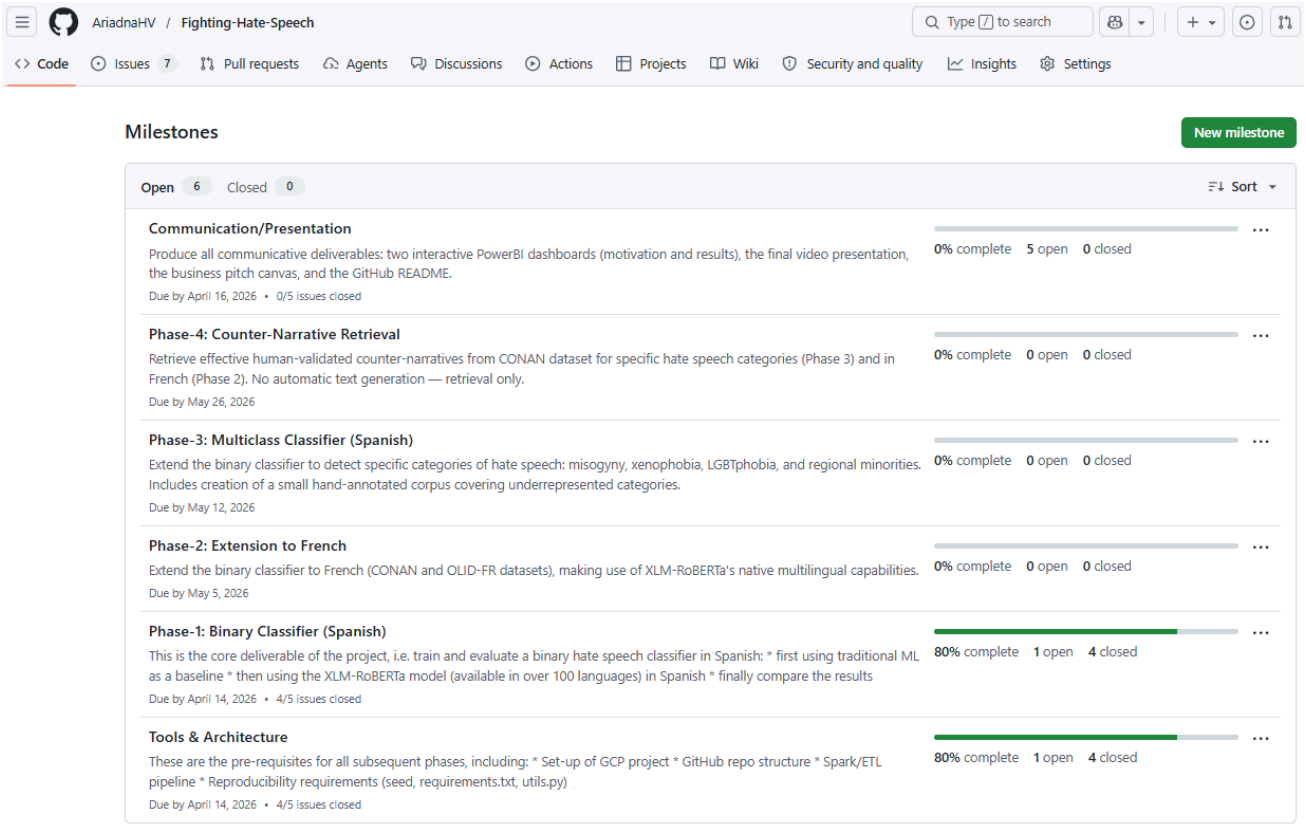


Y estas muestran las las subtareas concretas de las tareas *t&a - GitHub Repository Setup* y *1.3 - Machine Learning baseline*:





Las milestones, incluyendo las Fases 2, 3 y 4 que se completarán más allá de la entrega final del bootcamp, son las siguientes:



3. Datasets

El proyecto utiliza datasets existentes como base, complementados con un corpus propio anotado manualmente como aporte original en la Fase 3.

En español/castellano (fases 1 y 3):

- **Spanish Hate Speech Superset** (manueltonneau/spanish-hate-speech-Superset): superconjunto de 29.855 ejemplos que consolida todos los datasets públicos de discurso de odio en castellano disponibles hasta abril 2024 (mayoritariamente extraídos de Twitter), incluyendo HatEval (SemEval 2019). Las etiquetas ya están binarizadas (1 = odio, 0 = no odio). Se eligió este dataset en lugar de descargar HatEval de forma independiente para simplificar el pipeline y reducir el trabajo de preprocesamiento. Constituye el núcleo del entrenamiento en la Fase 1.
- **DETOXIS** (iberbench/iberbench_all, configuración iberlef-detoxis-toxicity_detection-2021-spanish): 3.463 comentarios tóxicos en español sobre inmigración, extraídos de noticias en línea. Utilizado en la Fase 1 para ampliar el corpus de entrenamiento.
- **HAHA** (iberbench/iberbench_all, configuración iberlef-haha-humor_detection-2021-spanish): 24.000 ejemplos de discurso de odio en español vehiculado a través del humor, incluyendo contenido LGTB-fóbico. Utilizado en las Fases 1 y 3.
- **OffComEs**: lenguaje ofensivo en redes sociales, incluyendo contenido LGTB-fóbico. Reservado para la Fase 3 de clasificación multiclase.

En francés (fase 2):

- **CONAN**: pares de discurso de odio y contranarraciones validadas por ONGs, disponible en francés, inglés e italiano. Exclusivamente utilizado con los ejemplos de francés en las Fases 2 y 4.
- **OLID-FR**: tweets ofensivos en francés con tres niveles de granularidad. Utilizado en la Fase 2.

Corpus propio: Se construirá un corpus anotado manualmente con 200-400 comentarios extraídos de medios de comunicación españoles mediante scraping, centrado en categorías infrarrepresentadas en los datasets existentes: minorías regionales (catalanes, vascos, gallegos) y contenido LGTB-fóbico. La anotación seguirá el marco de definiciones de la ONU y el Código de Conducta de la UE, y se medirá el acuerdo entre anotadores como control de calidad. Los nombres de usuario se eliminarán para cumplir con el RGPD.

Nota sobre los datasets descargados:

Los tres datasets descargados en la Fase 1 (Spanish Hate Speech Superset, DETOXIS y HAHA) no incluyen splits de validación y test predefinidos. Estas particiones se crearán durante la fase de preprocesamiento, con estratificación por clase para mantener la distribución de etiquetas en cada split.

Además, como el Superset afirma incluir todos los datasets públicos disponibles hasta abril 2024, es posible que haya solapamiento entre los tres. De hecho, mirando la descripción del Superset en HuggingFace, HAHA aparece explícitamente listado como una de sus fuentes. Esto significa que en el preprocesamiento será necesario ver si hay duplicados (parciales o totales).

4. Configuración del repositorio GitHub

El repositorio del proyecto está alojado en GitHub bajo el nombre *SinOdio/SansHaine*. La estructura del repositorio sigue las buenas prácticas de proyectos de ciencia de datos, con los siguientes elementos ya definidos desde el inicio del proyecto:

- **README.md**: documentación general del proyecto, a completar al final
- **.gitignore**: configurado para Python y Jupyter Notebooks, excluyendo credenciales y archivos temporales
- **requirements.txt**: lista de dependencias y versiones para garantizar la reproducibilidad del entorno
- **utils.py**: módulo de funciones compartidas, incluyendo la configuración de la semilla global para reproducibilidad de resultados
- **Notebooks**: organizados por fase del proyecto

La semilla global del proyecto es ***SEED = 42***, aplicada de forma consistente a todas las librerías utilizadas (Python, NumPy, PyTorch) mediante la función *set_seed()* definida en *utils.py*.

B. Arquitectura e infraestructura

1. Configuración del proyecto en GCP

Para el despliegue en la nube se ha utilizado Google Cloud Platform (GCP), renombrando el proyecto existente como *SinOdio/SansHaine*. Dado que la política de seguridad de la organización de GCP deshabilita la creación de claves de cuenta de servicio (iam.disableServiceAccountKeyCreation), se optó por utilizar **Application Default Credentials (ADC)** como método de autentificación. Este método es la alternativa recomendada por Google y elimina los riesgos de seguridad asociados a la gestión manual de claves.

Se configuraron los siguientes permisos IAM para la cuenta de servicio sinodio-sa:

- **Storage Admin**: lectura y escritura en Cloud Storage
- **BigQuery Admin**: lectura y escritura en BigQuery
- **Vertex AI User**: ejecución de trabajos de entrenamiento de modelos

2. Control de costes

Se configuró una alerta de presupuesto mensual de **50€** en la sección *Budgets & Alerts* de la consola de facturación, con notificaciones automáticas por correo electrónico al alcanzar el 50%, 90% y 100% del presupuesto. Durante el período de desarrollo, el proyecto opera bajo la cuenta de prueba gratuita de GCP, que incluye 300$ de crédito sin cargo a la tarjeta de crédito.

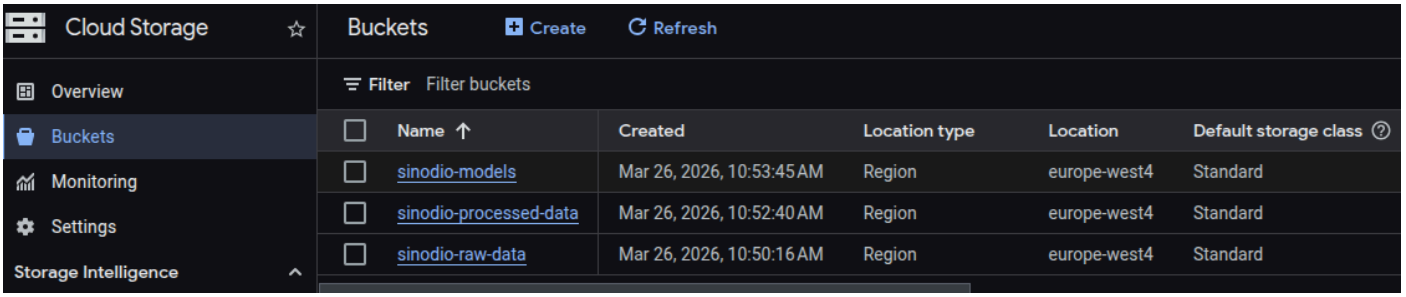
3. Cloud Storage

Se crearon tres buckets de almacenamiento en Cloud Storage con la siguiente configuración:

Bucket	**Propósito**
sinodio-raw-data	Almacenamiento de datasets originales descargados
sinodio-processed-data	Almacenamiento de datos preprocesados y limpios
sinodio-models	Almacenamiento de modelos entrenados

Configuración común a los tres buckets:

- **Región**: europe-west4 (Países Bajos). Se utiliza una región única para minimizar costes de transferencia y latencia entre servicios
- **Clase de almacenamiento**: Standard (adecuada para acceso frecuente durante el desarrollo)
- **Control de acceso**: Uniforme (permisos consistentes sobre todos los objetos del bucket)
- **Acceso público**: Deshabilitado (los datos de discurso de odio no deben ser accesibles públicamente)



4. BigQuery

Se ha creado un dataset en BigQuery bajo el nombre *sinodio_results*, configurado en la región europe-west4 para mantener consistencia con los buckets de **Cloud Storage** y evitar latencia y costes de transferencia de datos entre regiones. Este dataset actuará como repositorio central de los resultados del modelo, y estará conectado directamente a PowerBI mediante el conector nativo de GCP para la visualización de métricas e indicadores del proyecto.

Las tablas dentro del dataset se crearán de forma incremental a medida que el código genere resultados en cada fase del proyecto.

C. Configuración del repositorio y reproducibilidad

1. Repositorio GitHub

El repositorio del proyecto está alojado en GitHub bajo el nombre *Fighting-Hate-Speech*. La estructura de carpetas sigue las buenas prácticas de proyectos de ciencia de datos, tal y como lo hemos practicado en los diferentes módulos del bootcamp:

```
Fighting-Hate-Speech/
├── data/
│   ├── raw/
│   ├── processed/
│   └── annotated/
├── notebooks/
│   ├── phase1/
│   ├── phase2/
│   ├── phase3/
│   └── phase4/
├── sinodio_api/
│   ├── main.py
│   ├── Dockerfile
│   └── requirements_api.txt
├── src/
│   └── utils.py
├── models/
├── reports/
├── .gitignore
├── requirements.txt
└── README.md
```

Las carpetas vacías se mantienen bajo control de versiones mediante archivos *.gitkeep*. El archivo *.gitignore* está configurado para excluir archivos temporales de Python y Jupyter Notebooks, credenciales de GCP en formato *.json*, el contenido de las carpetas *data/* y *models/* (almacenados en GCP y no en GitHub), el entorno virtual *venv/*, y archivos de sistema operativo como *.DS_Store*.

El seguimiento de tareas se realiza mediante **GitHub Projects**, con un tablero de tres columnas (*To Do, In Progress, Done*) y seis milestones que corresponden a las fases del proyecto. Cada *issue* tiene asignada una etiqueta por fase para facilitar el filtrado.

2. Reproducibilidad

Para garantizar la reproducibilidad del proyecto se han tomado las siguientes medidas:

a) Entorno virtual

Se ha creado un entorno virtual con `python3 -m venv venv` para aislar las dependencias del proyecto del resto del sistema. El entorno virtual no se sube a GitHub.

b) Dependencias

Se han instalado las siguientes librerías dentro del entorno virtual:

Librería	Propósito
transformers	Carga y fine-tuning de XLM-RoBERTa
torch	Deep learning
datasets	Carga de datasets desde HuggingFace
scikit-learn	Modelos tradicionales de ML y métricas de evaluación
pandas	Manipulación de tablas
numpy	Operaciones numéricas
google-cloud-storage	Lectura y escritura en Cloud Storage
google-cloud-bigquery	Lectura y escritura en BigQuery
gradio	Interfaz de demostración del modelo
jupyter	Entorno de desarrollo en notebooks

El archivo *requirements.txt* se ha generado automáticamente con el comando:

```
pip freeze > requirements.txt
```

Este archivo captura el nombre y la versión exacta de cada librería instalada, garantizando que cualquier persona que clone el repositorio pueda reproducir el entorno de desarrollo con:

```
pip install -r requirements.txt
```

c) Semilla global

Se ha definido una semilla global `SEED = 42`, aplicada de forma consistente a todas las librerías que introducen aleatoriedad en el proceso de entrenamiento. Su implementación se encuentra en *utils.py*, descrita en la sección siguiente.

3. utils.py

Se ha creado el módulo *src/utils.py* como repositorio central de funciones compartidas entre los distintos notebooks del proyecto. En esta fase inicial contiene la implementación de la semilla global y las funciones base que se desarrollarán en fases posteriores:

```
import random
import numpy as np
import torch

SEED = 42

def set_seed(seed=SEED):
    """Set global seed for reproducibility across all libraries."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

def load_data(filepath):
    """Load dataset from given filepath.
    To be implemented in Phase 1."""
    pass

def preprocess_text(text):
    """Clean and normalize input text.
    To be implemented in Phase 1."""
    pass

def evaluate_model(y_true, y_pred):
    """Return precision, recall and F1 score.
    To be implemented in Phase 1."""
    pass
```

La función *set_seed()* aplica la semilla de forma consistente a Python, NumPy y PyTorch, incluyendo la GPU si está disponible. Las funciones *load_data(), preprocess_text()* y *evaluate_model()* están justo definidas como placeholders para ser implementadas progresivamente a partir de la Fase 1.

La función ***set_seed()* se llamará al inicio de cada notebook del proyecto** para garantizar que los resultados sean reproducibles en cualquier entorno.

D. Infraestructura de datos

1. Adquisición de datos

Se creó el notebook *notebooks/phase1/01_data_ingestion.ipynb* para la descarga automatizada de los datasets desde ***HuggingFace*** y su almacenamiento en el bucket *sinodio-raw-data* de GCP. El proceso de descarga utiliza la librería *datasets* de *HuggingFace* y requiere autentificación mediante un token de acceso, gestionado a través de un archivo *.env* para evitar exponer credenciales en el repositorio.

Los datasets descargados son los siguientes:

Dataset	Fuente	Splits	Número de filas
spanish-hate-speech-Superset	manueltonneau	train	29.855
detoxis-toxicity_detection-2021	iberbench/iberbench_all	train	3.463
haha-humor_detection-2021	iberbench/iberbench_all	train	24.000

El dataset *spanish-hate-speech-Superset* es un superconjunto que consolida todos los datasets públicos de discurso de odio en español disponibles a abril de 2024, incluyendo *HatEval* (*SemEval 2019*), con las etiquetas ya binarizadas (1 = odio, 0 = no odio). Se eligió este dataset en lugar de descargar HatEval de forma independiente para simplificar el pipeline y reducir el trabajo de preprocesamiento.

Los tres datasets se almacenan en formato CSV en el bucket *sinodio-raw-data*, organizados en subcarpetas por nombre de dataset. Dado que ninguno de los datasets incluye splits de validación y test predefinidos, estas particiones se crearán durante la fase de preprocesamiento.

2. Gestión de credenciales y seguridad

Durante el proceso de configuración surgió un incidente menor de seguridad: un token de autentificación de HuggingFace fue incluido accidentalmente en el código del notebook y detectado por el sistema de seguridad de GitHub, que bloqueó el push al repositorio remoto. El token fue borrado inmediatamente y sustituído por uno nuevo. Este incidente, menor ya que no significaba la pérdida de información crítica o con costes económicos, ilustra la importancia de gestionar las credenciales correctamente desde el inicio del proyecto, y queda documentado aquí como parte de las lecciones aprendidas.

En este caso, como medida correctiva, las credenciales se gestionan mediante un archivo *.env* que contiene el token de HuggingFace y que está excluido del repositorio a través del *.gitignore*. El notebook carga el token en tiempo de ejecución mediante la librería *python-dotenv*:

```
from dotenv import load_dotenv
import os
load_dotenv()
from huggingface_hub import login
login(token=os.getenv("HUGGINGFACE_TOKEN"))
```

E. Análisis Exploratorio de Datos (EDA)

El análisis exploratorio se llevó a cabo en el notebook *notebooks/phase1/02_eda.ipynb* y tuvo como objetivo comprender la estructura, calidad y características lingüísticas de los tres datasets antes de proceder al preprocesamiento y modelado.

1. Estructura y tipos de datos

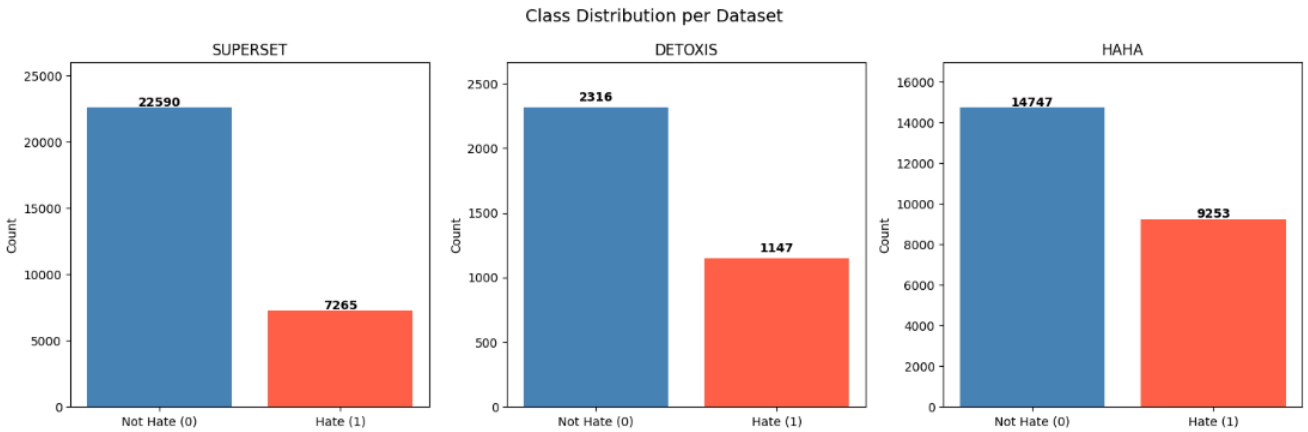
Los tres datasets presentan estructuras diferentes, reflejo de sus distintos orígenes:

Dataset	Filas	Columnas	Columna de texto	Columna de etiqueta
Superset	29.855	7	text	labels (float)
DETOXIS	3.463	3	text	label (int)
HAHA	24.000	3	text	label (int)

El Superset contiene columnas adicionales ("source", "dataset", "nb_annotators", "tweet_id", "post_author_country_location") que no se utilizarán en el modelado pero que aportan información contextual relevante. En particular, la columna "source" confirma que la totalidad de los textos del Superset provienen de Twitter, lo que implica un sesgo de plataforma importante: el lenguaje de Twitter es informal, breve, y hace uso frecuente de menciones ("@USER"), hashtags y abreviaciones. La columna "labels" es de tipo *float* en el Superset frente a *int* en los otros dos datasets, lo que deberá estandarizarse durante el preprocesamiento.

2. Distribución de clases

En el notebook se analiza la distribución de clases y se generan histogramas para cada uno de los datasets:



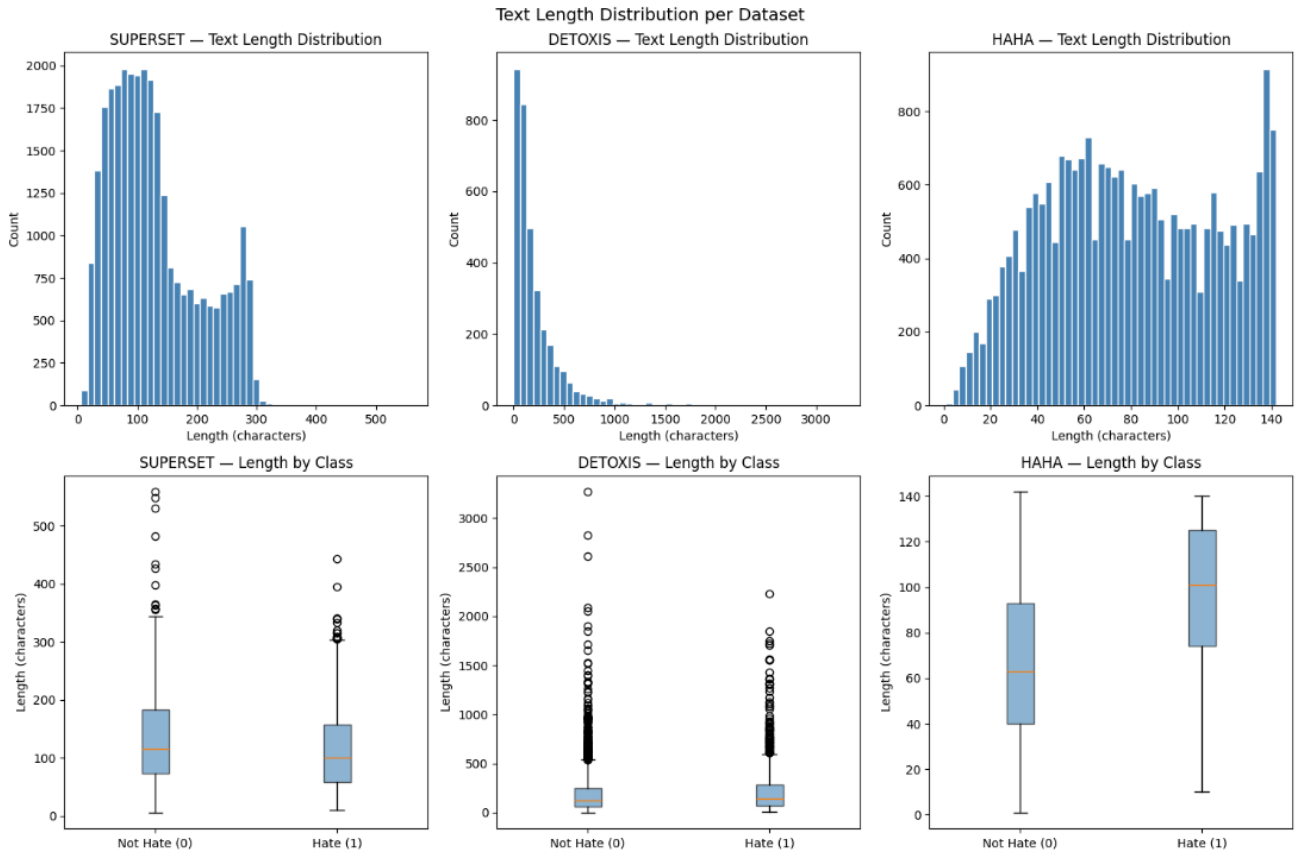
Los tres datasets presentan un desequilibrio de clases moderado, con la clase no-odio dominante en todos los casos:

Dataset	No odio (0)	Odio (1)	% No odio (0)	% Odio (1)
Superset	22.590	7.265	75,7%	24,3%
DETOXIS	2.316	1.147	66,9%	33,1%
HAHA	14.747	9.253	61,4%	38,6%

El desequilibrio es más pronunciado en el Superset (76% / 24%), mientras que HAHA es el más equilibrado (61% / 39%). Al fusionar los tres datasets, la distribución global se aproximará a la del Superset, por ser el más voluminoso. Este desequilibrio es realista, ya que en datos reales de redes sociales el discurso de odio es genuinamente minoritario, pero tiene implicaciones para el modelado: se utilizará *class_weight='balanced'* en los modelos de ML tradicional y se monitorizará el F1 macro en lugar de la exactitud (*accuracy*) como métrica principal de evaluación, siguiendo el mismo criterio aplicado en el módulo de NLP del bootcamp.

3. Distribución de longitud de textos

Se analiza la longitud de los textos en caracteres, tanto a nivel global como desglosada por clase, y se producen los siguientes histogramas y diagramas de cajas para ilustrarlo:



La tabla siguiente resume la información más significativa:

Dataset	Media global	Mediana global	Máximo	Media no-odio	Media odio
Superset	129,3	113,0	559	132,7	118,7
DETOXIS	206,7	127,0	3.270	199,3	221,5
HAHA	79,7	77,0	142	68,1	98,2

Los tres datasets presentan patrones distintos en la relación entre longitud y clase. En el Superset, el discurso de odio tiende a ser ligeramente más corto que el no-odio, lo cual es consistente con el registro de Twitter donde el odio suele expresarse de forma directa y concisa. En DETOXIS, el patrón se invierte: los comentarios de odio son más largos, lo que refleja el carácter argumentativo de los comentarios de noticias, donde los usuarios elaboran sus posiciones. En HAHA, la diferencia es la más pronunciada: el humor ofensivo es sustancialmente más largo (media 98 vs 68 caracteres), posiblemente porque los chistes ofensivos requieren más contexto narrativo que el humor neutro.

La siguiente captura de pantalla muestra los textos más largos de cada dataset:

```
============================================================
SUPERSET — Longest text (559 characters):
============================================================
@USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER
@USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER @USER
@USER @USER @USER la migración es argumento?\nYa tardabas !!\n.. si tan mal están y tan desnutrios están por que esos países tan prósperos
no acogen y dan trabajo y riquezas a los migrantes?\n\n\nO simplemente los migrantes huyen de guerras y bloqueos que sufren de esos paise
s. LINK

============================================================
DETOXIS — Longest text (3270 characters):
============================================================
en el primer enlace que aporta no dice nada de eso. De hecho pide lo que ya hizo el gobierno cuando desplegó a la UME. ¿Es el gobierno act
ual fascista por ello? TOdas juntas podrían convertirte en fascista si añadiera otros principios fundamentales de tal ideología, pero resu
lta que todas ellas, en mayor o menor medida, son aplicables a muchos otros partidos. De hecho, como ya le he señalado antes, ni en vox se
dan todas ellas juntas. Muy bien. No me pienso leer las cien medidas pero le he echado un ojo por encima y no veo qué puede ser eso que lo
s convierta en fascista. Si es usted tan amable de concretar un poco. La 59 niega el acceso gratuito a los inmigrantes irregulares, como l
a abrumadora mayoría de países de mundo (excluyendo urgencias sanitarias) y a los regularizados que tengan menos de 10 años de permanenci
a. Eso es como decir que lleven 10 años pagando impuestos, ni tan siquiera diez años de regularizados, sino que lleven 10 años aportando a
las arcas en igualdad de condiciones que los demás ciudadanos. Podrá parecerle una medida desproporcionada o estúpida pero desde luego no
es una medida racista ni xenófoba: no indica que se les prohiba por raza o por extranjeros sino por aportación al fondo común. Contra el a
cuerdo Schengen también estaba pàblo iglesias y podemos. Es más contra la UE. NO sé si habrán cambiado de opinión. Lo que sí sé es que eso
no les convierte en fascistas. Es más el argumento es similar: nos quita soberanía, un ejemplo es la soberanía monetaria como cuando el he
rmano del ministro garzón pretendía solucionar los problemas de deuda con la estrategia andy y lucas. Estar contra el matrimonio homosexua
l no implica estar contra los homosexuales. Yo estoy contra el matrimonio, todo, pero no estoy contra el amor, son categorías relacionadas
pero no idénticas. Están contra el matrimonio homosexual porque consideran que es una institución religiosa (e históricamente y tradiciona
lmente lo es). Es como pretender una comunión laica o un bautizo laico; legítimo, pero absurdo. EL estado en lo único que debería interven
ir respecto a la unión de dos personas en registrarlo en un documento. Así de aséptico. Y estar en contra del matrimonio homosexual no te
convierte en fascista, estar contra la homosexualidad tampoco (insisto con castro). Vale, con su frase: El rechazo a la homosexualidad les
viene de su faceta ultrarreligiosa, otro aspecto que les une a otras grandes fascistas de la historia. Y ya llevamos unos cuantos. se ha c
ubierto usted de gloria. No, el fascismo no es ultrarreligioso. De hecho el fascismo deslegitima a la religión porque es uno de sus grande
s competidores como directores de la sociedad; el fascismo pretende fagotizar toda la vida social, religión incluida. COmo el comunismo, p
or cierto: nada más allá del estado. Lo que pasa es que confunde usted nacional catolicismo con fascismo. COnfusión habitual en españa per
o no por ello menos errónea. Yo dudo. Es parte de mi idiosincrasia. TIene cosas buenas y cosas malas. Pero es como soy y no puedo evitarl
o. Y como dudo, investigo. Respecto a este asunto le invito a que dude usted. A que se pregunte qué es el fascismo, acuda a las fuentes or
iginales, y contraste con sus ideas preconcebidas. Apuesto a que se lleva más de una sorpresa

============================================================
HAHA — Longest text (142 characters):
============================================================
Viendo pal techo como cada noche, considerando seriamente el suicidio.

[======]
| ∧_∧    |
| (  ･ω･`) |
|ノ ￣￣ヽ |
|   ≧ .|
|        |
ヽ____ソ
```

El texto más largo del Superset (559 caracteres) consiste casi en su totalidad en menciones "@USER" anonimizadas, lo que infla artificialmente la longitud. El texto más largo de DETOXIS (3.270 caracteres) es un comentario argumentativo extenso sobre política española. El texto más

largo de HAHA (142 caracteres) contiene una referencia suicida, seguida de arte ASCII, lo que recuerda que los datasets de discurso de odio pueden estar relacionados con contenido sensible.

Desde el punto de vista del modelado, la longitud máxima de DETOXIS es especialmente relevante: XLM-RoBERTa tiene un límite de 512 tokens, y textos de 3.270 caracteres corresponden aproximadamente a 650-800 tokens, superando dicho límite. La estrategia de truncación se definirá en el notebook de preprocesamiento. Las implicaciones de esta truncación para la Fase 4 de recuperación de contranarraciones se discuten en la sección de limitaciones y trabajo futuro.

4. Valores nulos, duplicados y textos vacíos

Valores nulos:

Las columnas relevantes para el modelado ("text" y "label" o "labels" según el dataset) no contienen valores nulos en ninguno de los tres datasets. El Superset presenta nulos únicamente en columnas no utilizadas para el modelado: "source" (304 nulos) y "tweet_id" (10.589 nulos).

Duplicados:

Dataset	Duplicados exactos	Textos duplicados	Etiquetas conflictivas
Superset	87	198	5
DETOXIS	17	18	1
HAHA	0	0	0

Los 198 textos duplicados del Superset son consistentes con los 29.657 valores únicos de texto sobre 29.855 filas totales observados en la exploración inicial. Los 6 casos de etiquetas conflictivas (mismo texto etiquetado como odio por una fuente y como no-odio por otra) reflejan la ambigüedad inherente a la anotación de discurso de odio y se resolverán durante el preprocesamiento eliminando estos ejemplos, ya que su ambigüedad los hace poco útiles para el entrenamiento.

Solapamiento entre datasets:

Comparación	Textos en común
DETOXIS y Superset	0
HAHA y Superset	0
DETOXIS y HAHA	2

La ausencia de solapamiento entre el Superset y los otros dos datasets es un resultado muy favorable: garantiza que no habrá fuga de datos (*data leakage*) entre conjuntos de entrenamiento y test cuando se fusionen los tres datasets.

Los 2 textos compartidos entre DETOXIS y HAHA se eliminarán de uno de ellos durante el preprocesamiento.

Textos vacíos o muy cortos:

Se identifican 9 textos con menos de 3 caracteres: 7 en DETOXIS ("X", "V", "xD", "ok", "+1", "+1", "+1") y 2 en HAHA (":(", "."). Todos pertenecen a la clase no-odio y se eliminarán durante el preprocesamiento por no contener contenido lingüístico significativo para la clasificación. Esto no tendría que afectar negativamente el modelado ya que pertenecen a la clase mayoritaria de no-odio.

5. Análisis de menciones ("@USER")

Se analiza el número de menciones "@USER" por texto en el Superset para investigar si el comportamiento de mención masiva, una táctica utilizada en Twitter, podría ser un indicador de discurso de odio.

Clase	Media @USER	Mediana @USER	Máximo @USER
No odio (0)	0,89	1,0	50
Odio (1)	0,97	1,0	32

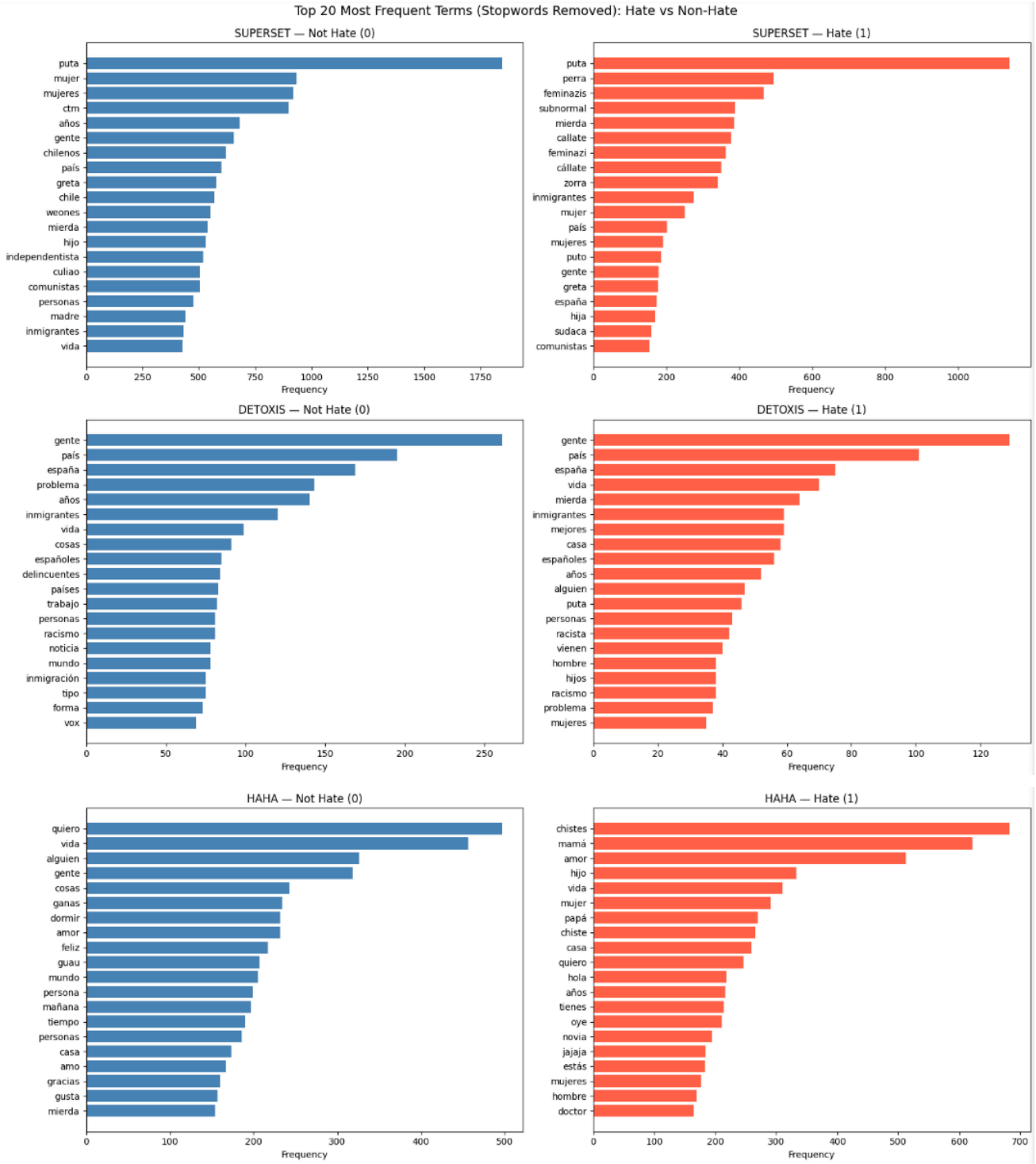
Los resultados no muestran diferencia significativa entre clases: ambas tienen la misma mediana y medias prácticamente idénticas. La hipótesis de que el acoso coordinado mediante menciones masivas sería un indicador discriminante no se confirma en este dataset. Esto sugiere que los patrones de acoso están codificados en el contenido semántico del texto más que en sus características estructurales. Este hecho refuerza la elección de XLM-RoBERTa como modelo principal: al utilizar un mecanismo de atención que captura relaciones contextuales entre todas las palabras del texto simultáneamente, será capaz de detectar patrones sutiles de discurso de odio que métricas estructurales como el conteo de menciones no pueden capturar.

6. Análisis de vocabulario por clase

Se efectúa una limpieza de datos consistente en:

- Eliminar stopwords en castellano (usando el modelo "es_core_news_sm" de spaCy)
- Eliminar dígitos, URLs, menciones "@USER"
- Eliminar los artefactos de Twitter "link", "rt" y "amp", que son códigos que provienen del scrapeado de páginas web:
    - "link" sustituye un enlace
    - "rt" es la abreviatura de *retweet* (indicando que el texto es una republicación de otro usuario)
    - "amp" es un residuo del símbolo "&", que en HTML se escribe "&" (después de quitar la puntuación queda como "amp").

Luego, se analizan los términos más frecuentes en cada clase y se representan con barras horizontales (ver página siguiente).



<u>Superset (Twitter)</u>:

El vocabulario de odio está dominado por insultos explícitos y términos misóginos: "puta", "perra", "feminazis", "subnormal", "mierda", "zorra". El vocabulario de no-odio es más heterogéneo e incluye profanidad chilena de uso coloquial: "ctm" (abreviatura de *conchetumadre*), "weones" (derivado de *huevones*) y "culiao" (derivado de *culiado*). Estos términos son expresiones vulgares de uso cotidiano en el español chileno que no implican necesariamente intención ofensiva, lo que confirma el fuerte sesgo latinoamericano, más específicamente chileno, del Superset. La aparición de "independentista" en la clase no-odio sugiere la presencia de contenido relacionado con los movimientos de independencia catalán o vasco sin incitar al odio.

DETOXIS (comentarios de noticias):

Ambas clases comparten un vocabulario temático similar centrado en la inmigración ("inmigrantes", "españoles", "españa", "país"), lo que refleja que todos los textos del dataset tratan el mismo tema. La clase de odio se distingue por términos como "mierda", "racista", "racismo", "hombre" y "mujeres", mientras que la clase de no-odio presenta vocabulario más analítico: "problema", "trabajo", "mundo", "forma". La presencia de "vox" en la clase de no-odio indica que hay comentarios que mencionan al partido político en un contexto factual sin incitar al odio.

HAHA (humor):

Es el dataset con el vocabulario más distintivo entre clases. La clase de no-odio presenta un registro conversacional cotidiano: "quiero", "vida", "feliz", "dormir", "amor", "gracias". La clase de odio se estructura entorno a relaciones familiares y marcadores de humor: "chistes", "mamá", "papá", "hijo", "chiste", "jajaja". Esto confirma que el humor ofensivo en este dataset no se expresa mediante insultos explícitos sino mediante una estructura narrativa de chiste con referencias familiares, lo que representa un reto específico para los clasificadores basados en vocabulario.

A continuación se presentan los 10 términos más frecuentes por clase en cada dataset en forma de lista (captura de pantalla del notebook):

```
SUPERSET — Top 10 filtered terms:
  Not Hate: ['puta', 'mujer', 'mujeres', 'ctm', 'años', 'gente', 'chilenos', 'país', 'greta', 'chile']
  Hate:     ['puta', 'perra', 'feminazis', 'subnormal', 'mierda', 'callate', 'feminazi', 'cállate', 'zorra', 'inmigrantes']

DETOXIS — Top 10 filtered terms:
  Not Hate: ['gente', 'país', 'españa', 'problema', 'años', 'inmigrantes', 'vida', 'cosas', 'españoles', 'delincuentes']
  Hate:     ['gente', 'país', 'españa', 'vida', 'mierda', 'inmigrantes', 'mejores', 'casa', 'españoles', 'años']

HAHA — Top 10 filtered terms:
  Not Hate: ['quiero', 'vida', 'alguien', 'gente', 'cosas', 'ganas', 'dormir', 'amor', 'feliz', 'guau']
  Hate:     ['chistes', 'mamá', 'amor', 'hijo', 'vida', 'mujer', 'papá', 'chiste', 'casa', 'quiero']
```

Observación: Como no se aplicó lematización en esta fase del análisis, aparecen algunas formas duplicadas como "feminazi" y "feminazis". Conviene destacar que, en el caso de neologismos (como probablemente “feminazi”), spaCy no reduciría correctamente las palabras que no forman parte de su vocabulario. Esto es una limitación general de las herramientas estándar de procesamiento de lenguaje natural (NLP) aplicadas a datos de discurso de odio, que contienen vocabulario no estándar, jerga e incluso lenguaje codificado deliberadamente para evadir la detección y el bloqueo automático.

Implicación general:

Los tres datasets presentan perfiles de vocabulario de odio muy distintos: insultos explícitos en el Superset, discurso xenófobo en DETOXIS, y humor ofensivo en HAHA. Un modelo entrenado únicamente en uno de ellos probablemente fallaría en los otros dos. La heterogeneidad resultante de combinar los tres datasets obliga al modelo a aprender representaciones más generalizables del discurso de odio, más allá de un único registro o grupo objetivo.

7. Conclusiones del EDA y decisiones para el preprocesamiento

El análisis exploratorio revela que los datos son de buena calidad general, con ausencia de valores nulos en las columnas relevantes y un nivel de duplicación manejable. Las principales acciones de preprocesamiento identificadas son:

- Eliminar duplicados exactos (87 en Superset, 17 en DETOXIS)
- Eliminar los 6 casos de etiquetas conflictivas (mismo texto con etiquetas contradictorias)
- Eliminar los 2 textos compartidos entre DETOXIS y HAHA
- Eliminar los 9 textos con menos de 3 caracteres
- Estandarizar nombres de columnas y tipos de datos entre los tres datasets
- Eliminar tokens "@USER", URLs y artefactos de Twitter durante la limpieza de texto
- Definir una estrategia de truncación para textos que superen los 512 tokens de XLM-RoBERTa
- Crear los splits de entrenamiento, validación y test con estratificación por clase

Desde el punto de vista lingüístico, el análisis exploratorio de los datos pone de manifiesto la diversidad dialectal y de registro del corpus, con presencia significativa de español latinoamericano (especialmente chileno) junto a español peninsular (castellano), y la prevalencia de vocabulario no estándar en el discurso de odio, lo que justifica el uso de un modelo transformer multilingüe pre-entrenado en lugar de enfoques basados exclusivamente en características léxicas.

F. Preprocesamiento y ETL

El preprocesamiento se llevó a cabo en el notebook *notebooks/phase1/03_preprocessing.ipynb* y tuvo como objetivo transformar los tres datasets “crudos” (*raw* en inglés) en un corpus unificado, limpio y listo para el entrenamiento de modelos. El *pipeline* sigue las acciones identificadas durante el EDA y produce dos versiones del texto adaptadas a las necesidades de los distintos modelos.

1. Carga de datos desde GCP

Los tres datasets se cargan directamente desde el bucket *sinodio-raw-data* de Google Cloud Storage en formato CSV, tal como fueron almacenados durante la fase de adquisición de datos:

- *spanish_hate_superset/train.csv* → 29.855 filas, 7 columnas
- *detoxis/train.csv* → 3.463 filas, 3 columnas
- *haha/train.csv* → 24.000 filas, 3 columnas

2. Estandarización de columnas y tipos de datos

Los tres datasets presentan estructuras distintas que se unifican en un esquema común de tres columnas: “text”, “label” y “source”. En el Superset, la columna “labels” se renombra a “label” y se convierte de *float* a *int64*. En los tres datasets, los valores de etiqueta son binarios: 0 (no odio) y 1 (odio).

3. Eliminación de duplicados, etiquetas conflictivas y textos muy cortos

Se aplica un pipeline de limpieza que elimina duplicados exactos (mismo texto y misma etiqueta) dentro de cada dataset, textos que aparecen con etiquetas contradictorias en el mismo dataset, solapamientos entre datasets y los textos con menos de 3 caracteres (7 en total). Este último paso se hace para evitar posibles errores resultantes de la transformación de datos, al considerarse que no contienen información lingüística significativa para la clasificación. Además, ninguno de los textos eliminados pertenecen a la clase de odio, así que esto no afecta negativamente a la clase minoritaria.

Los textos eliminados son:

- en DETOXIS: “X”, “V”, “xD”, “ok” y “+1”
- en HAHA: ”:(“ y “.”

Después de este pipeline de limpieza, los tamaños resultantes son:

- Superset: 29.652 filas, 3 columnas
- DETOXIS: 3.439 filas, 3 columnas
- HAHA: 23.997 filas, 3 columnas

4. Limpieza de texto

Se definen dos funciones de limpieza que producen dos versiones del texto con distinto nivel de procesamiento, adaptadas a las necesidades de cada familia de modelos:

1) La función clean_text_minimal, enfocada a los datos que serán entrenados con XLM-RoBERTa

Como XLM-RoBERTa fue preentrenado sobre texto no procesado e incluye su propio tokenizador que maneja puntuación, mayúsculas y estructura de la oración, una limpieza excesiva podría degradar su rendimiento. Por esta razón, la función clean_text_minimal se limita a eliminar los artefactos que no aportan información semántica:

- Menciones "@USER" → sustituidas por espacio
- URLs → sustituidas por espacio
- Marcador de retweet “rt” → sustituido por espacio (“rt” es la abreviatura de *retweet* e indica que el texto es una republicación de otro usuario)
- Artefacto HTML “amp” → sustituido por espacio (“amp” es el residuo del símbolo “&”, que se escribe “&” en HTML, y aparece con el scraping)
- Espacios múltiples → normalizados a un único espacio

Cabe decir que en un principio simplemente se eliminaron los ítems (en vez de sustituirlos por un espacio), pero esta técnica dió lugar a algunos casos de concatenaciones, como “*gobiernocon*” en vez de “*gobierno con*”. Para evitar este problema, se reemplazaron todos los ítems por un espacio y al final se normalizaron los espacios múltiples a un único espacio.

El texto resultante de esta limpieza queda registrado en la columna “text_clean” del DataFrame.

2) La función clean_text_tfidf, una limpieza más exhaustiva para la utilización por los modelos de Machine Learning tradicional.

TF-IDF representa el texto como un vector de frecuencias de vocabulario. Para reducir el ruido y mejorar la discriminación entre clases, la función clean_text_tfidf aplica primero la función clean_text_minimal explicada más arriba, y después hace lo siguiente:

- Conversión a minúsculas
- Eliminación de puntuación → sustituida por espacio (para evitar concatenaciones como *"gobernación...una"* → *"gobernaciónuna"*)
- Eliminación de dígitos → sustituidos por espacio
- Normalización de espacios múltiples
- Eliminación de stopwords en español (modelo es_core_news_sm de spaCy, complementado con artefactos adicionales: “link”, “rt”, “user”, “http”, “https”, “amp”, “via”, aunque algunos de estos ya tendría que haber quedado eliminados con clean_text_minimal)
- Eliminación de tokens de menos de 3 caracteres

El texto resultante de esta limpieza queda registrado en la columna “text_tfidf” del DataFrame.

5. Fusión del corpus

Los tres datasets limpios se fusionan en un único corpus mediante `pd.concat`, y se reinician los índices. El corpus resultante contiene 57.088 textos, 5 columnas (“text”, “label”, “source”, “text_clean”, “text_tfidf”) y ningún valor nulo. La distribución de clases es 69,2% no odio (0) y 30,8% odio (1).

6. División en conjuntos de entrenamiento, validación y test

Se realiza una división estratificada por clase en dos pasos, usando `random_state=42` en ambos para garantizar la reproducibilidad:

1. Primera división: 15% para test, 85% restante para entrenamiento y validación
2. Segunda división: 15% del conjunto restante para validación, 85% para entrenamiento

La proporción resultante es aproximadamente 72,25% entrenamiento / 12,75% validación / 15% test. La estratificación garantiza que la distribución de clases se mantenga en los tres conjuntos, con un porcentaje de etiqueta de odio de 30,84% en entrenamiento, 30,84% en validación y 30,85% en test.

Los splits se guardan localmente en *data/processed/* en formato `Parquet`, que preserva los tipos de datos y se integra de forma nativa con GCP, BigQuery y Spark.

7. Almacenamiento y registro de métricas usando PySpark

Los tres splits en formato Parquet se suben al bucket *sinodio-processed-data* de Cloud Storage mediante el cliente `google-cloud-storage`, dentro de la carpeta *phase1/*.

A continuación se utiliza PySpark para calcular las métricas de preprocesamiento sobre los splits almacenados. Se optó por PySpark en lugar de Scala (utilizado en el módulo de Big Data Processing) por consistencia con el resto del pipeline del proyecto, desarrollado íntegramente en Python. PySpark es igualmente válido para este tipo de agregaciones y también se cubrió brevemente en el módulo. Las métricas calculadas incluyen el número de ejemplos por *split*, la distribución de clases, el porcentaje de discurso de odio y la longitud media del texto limpio.

Los resultados se escriben en BigQuery en la tabla *preprocessing_summary* dentro del dataset *sinodio_results*, que actúa como repositorio central de métricas y resultados del proyecto a lo largo de todas las fases.

G. Baseline de Machine Learning Tradicional

El entrenamiento del baseline de machine learning tradicional se llevó a cabo en el notebook *notebooks/phase1/04_baseline_ml.ipynb*. El objetivo de esta fase es establecer un nivel de rendimiento de referencia mediante modelos clásicos de NLP, que posteriormente será comparado con el modelo transformer XLM-RoBERTa en la fase de deep learning.

Este enfoque responde al requisito del proyecto de abordar el mismo problema con dos familias de técnicas distintas y comparar sus resultados.

1. Extracción de características: TF-IDF

La representación vectorial de los textos se realiza mediante TF-IDF (Term Frequency - Inverse Document Frequency), aplicado sobre la columna “text_tfidf” generada durante el preprocesamiento. TF-IDF asigna a cada token un peso que refleja su frecuencia en un documento concreto, penalizada por su frecuencia en el corpus global: los tokens frecuentes en un documento pero raros en el resto del corpus reciben pesos más altos, y de esta forma son más susceptibles de aportar una señal discriminativa al clasificador.

Los parámetros utilizados son:

- `ngram_range=(1,3)`: se incluyen unigramas, bigramas y trigramas para capturar expresiones relevantes para la detección de discurso de odio, como "cállate zorra" o "inmigrantes saltan valla"
- `max_features=150.000`: el vocabulario se establece en 150k tokens para contar con suficiente memoria para los n-gramas
- `min_df=2`: se excluyen tokens que aparecen en un único documento para reducir ruido
- `sublinear_tf=True`: se aplica normalización logarítmica a la frecuencia de término para amortiguar el efecto de repeticiones de un mismo token dentro de un documento

El vectorizador se ajusta exclusivamente sobre el conjunto de entrenamiento y se aplica posteriormente a validación y a test, evitando así la fuga de datos.

<u>Observación</u>: Cabe destacar que, aunque se definió un límite de vocabulario de 150k, el vocabulario real que se construyó a partir de los datos de entrenamiento con `min_df=2` resultó ser de 42k tokens (42.065 exactamente), incluyendo todos los n-gramas. Esto tiene cierto sentido porque los trigramas son mucho más escasos que los unigramas (la mayoría de combinaciones de tres palabras quedan eliminadas porque aparecen en un único documento) y porque las temáticas de los textos son reducidas, limitando la diversidad del vocabulario.

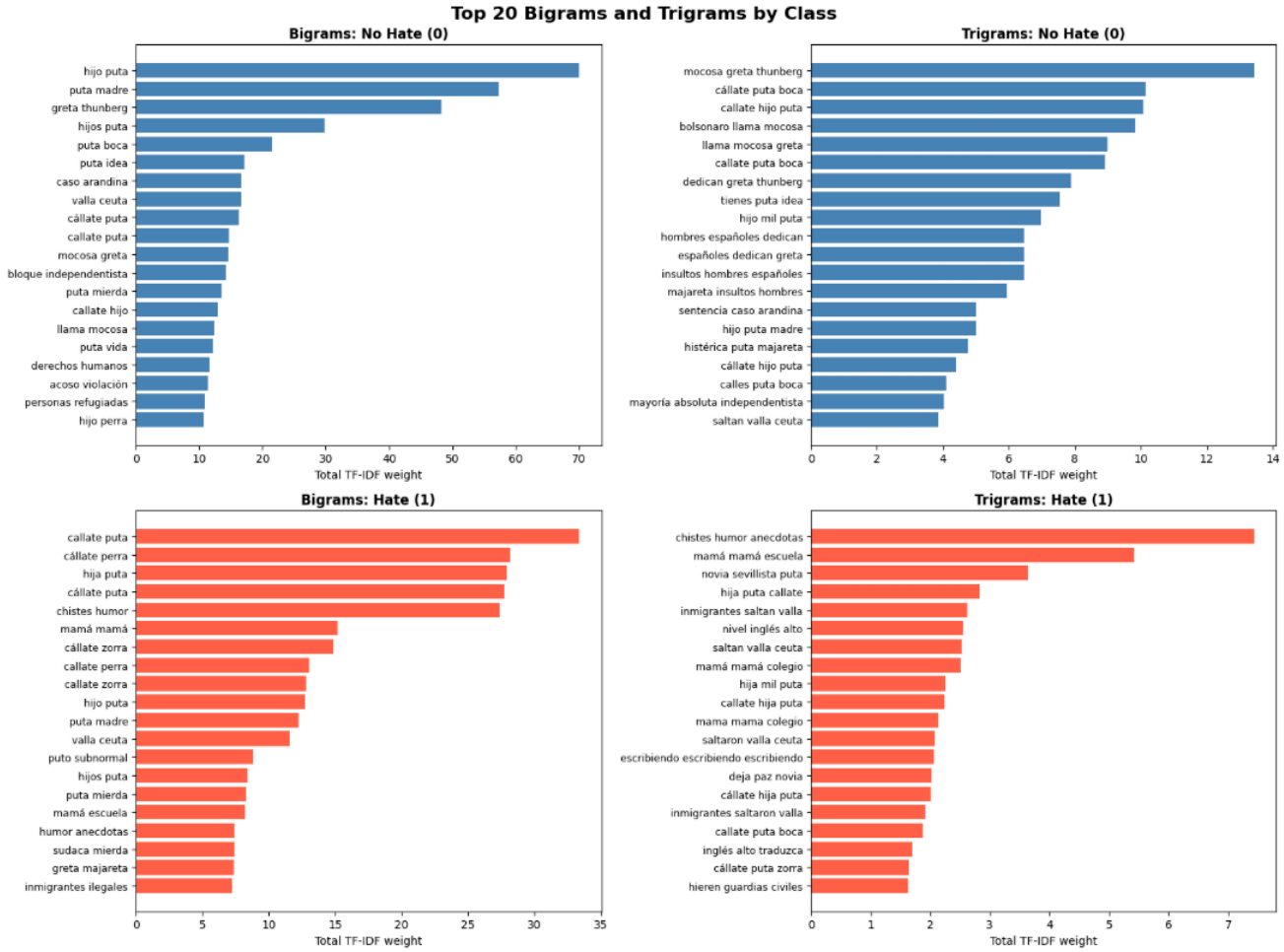
<u>Vías de mejora</u>:

- Una limitación conocida es que no se aplicó lematización durante el preprocesamiento, lo que implica que formas distintas de una misma palabra aparecen como tokens distintos (por ejemplo, "cállate" y "callate"). Esto representa una vía de mejora clara para este pipeline de ML tradicional en iteraciones futuras.
- El vectorizador TF-IDF y el clasificador se gestionaron como objetos independientes, lo que requirió aplicar manualmente la transformación al conjunto de validación y test tras ajustar el vectorizador sobre el entrenamiento. Una mejora recomendada para iteraciones futuras sería encapsular ambos pasos en un Pipeline de scikit-learn (`sklearn.pipeline.Pipeline`), que

encadena el preprocesamiento y el modelo en un único objeto serializable. Esto garantizaría la ausencia de fuga de datos de forma nativa, simplificaría el reentrenamiento sobre nuevos datos -por ejemplo, el corpus en francés previsto para la Fase 2- reduciéndolo a una única llamada .fit(), y permitiría serializar el pipeline completo como un único artefacto para su almacenamiento en Cloud Storage.

2. Análisis de n-gramas por clase

Se extraen y visualizan los 20 bigramas y trigramas más frecuentes por clase (odio y no odio) a partir del conjunto de entrenamiento.



El análisis revela perfiles de vocabulario muy distintos entre clases y entre datasets:

- En la clase de **no odio**, dominan expresiones coloquiales del español latinoamericano (especialmente chileno) como "hijo puta" y "puta madre", que en ese contexto no expresan intención ofensiva, junto con referencias a contenido político y de actualidad ("greta thunberg", "bloque independentista", "caso arandina").
- En la clase de **odio**, los bigramas más frecuentes son insultos directos ("cállate puta", "cállate perra", "hija puta"), y los trigramas reflejan claramente la estructura narrativa de los chistes ofensivos del dataset HAHA ("chistes humor anécdotas", "mamá mamá escuela") y contenido xenófobo del dataset DETOXIS ("inmigrantes saltan valla", "saltan valla ceuta").

Esta heterogeneidad entre datasets confirma el acierto de combinar los tres corpus: un modelo entrenado sobre uno solo de ellos probablemente no generalizaría bien a los otros dos registros.

3. Modelos entrenados

Se entrenan dos modelos con class_weight='balanced' para compensar el desequilibrio de clases (69% no odio / 31% odio), evaluando primero con parámetros por defecto y optimizando posteriormente con Optuna (ver sección siguiente).

Regresión Logística: La regresión logística es un clasificador lineal que estima la probabilidad de pertenencia a cada clase mediante una función sigmoide aplicada a una combinación lineal de las características TF-IDF. Se utilizó max_iter=1000 para aumentar el número máximo de iteraciones permitidas y reducir el riesgo de que el algoritmo no converja en el espacio de alta dimensionalidad de los vectores TF-IDF. El parámetro solver=liblinear especifica el algoritmo de optimización utilizado para ajustar los coeficientes del modelo; liblinear utiliza descenso de coordenadas y es especialmente adecuado para datos dispersos de alta dimensionalidad como los vectores TF-IDF.

Random Forest: Random Forest es un conjunto de árboles de decisión independientes (aquí, 176 después de la optimización con Optuna). Cada árbol es una serie de preguntas del tipo "¿contiene este texto la palabra ___?". Para garantizar la independencia entre árboles, cada uno se entrena sobre un subconjunto aleatorio de ejemplos y considera un subconjunto aleatorio de características TF-IDF en cada división. La predicción final se obtiene por votación mayoritaria entre todos los árboles: aunque cada árbol individual comete errores, los errores tienden a cancelarse entre sí al agregar las predicciones, y así el modelo resultante es más robusto que cualquier árbol individual. Se utilizó n_jobs=-1 para aprovechar todos los núcleos disponibles de CPU durante el entrenamiento, dado el coste computacional de construir múltiples árboles sobre un espacio de 42k características.

Los resultados de estos primeros entrenamientos con parámetros por defecto sobre el conjunto de validación fueron los siguientes:

Modelo	Accuracy	Precisión	Recall	F1 Macro
Regresión Logística	0,77	0,74	0,75	0,74
Random Forest	0,78	0,76	0,69	0,71

Observación: A pesar de ser un modelo más complejo, Random Forest obtiene un F1 macro inferior al de la Regresión Logística (0,71 vs 0,74). La diferencia clave está en el recall de la clase de odio: 0,46 en Random Forest frente a 0,68 en Regresión Logística, lo que indica que Random Forest es demasiado conservador a la hora de predecir discurso de odio. Este patrón parece ser típico de Random Forest aplicado a datos dispersos de alta dimensionalidad como los vectores TF-IDF,

donde los árboles de decisión no realizan divisiones adecuadas sobre características con muchos ceros. En el caso del módulo de Procesamiento de Lenguaje Natural, también observé que la Regresión Logística constituye una base más sólida para clasificación de texto (además de ser más eficiente), seguramente por esta razón.

4. Optimización de hiperparámetros con Optuna

Se utiliza Optuna para optimizar los hiperparámetros de ambos modelos, maximizando el F1 macro sobre el conjunto de validación (media no ponderada del F1 de cada clase, que da igual importancia al rendimiento sobre odio y no odio).

Regresión Logística (50 trials, timeout 10 minutos):

- Espacio de búsqueda: `C` (fuerza de regularización, log-uniforme entre 0.01 y 10) y `solver` (`liblinear` o `saga`)
- Mejores parámetros: `C=1.607`, `solver=liblinear`
- Mejora de F1 macro en validación: 0.740 → 0.7435

Random Forest (30 trials, timeout 60 minutos):

- Espacio de búsqueda: `n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features`
- Mejores parámetros: `n_estimators=176`, `max_depth=None`, `min_samples_split=9`, `min_samples_leaf=2`, `max_features=log2`
- Mejora de F1 macro en validación: 0.71 → 0.7402

5. Evaluación de los modelos ajustados

Se reentrenan ambos modelos con sus mejores parámetros y se evaluan sobre los conjuntos de validación y test. Los resultados se muestran en esta tabla:

Modelo	Split	Accuracy	Precisión	Recall	F1 Macro
Regresión Logística	val	0.776	0.738	0.749	0.743
Regresión Logística	test	0.775	0.739	0.754	0.745
Random Forest	val	0.778	0.740	0.741	0.740
Random Forest	test	0.782	0.745	0.743	0.744

La consistencia entre validación y test en ambos modelos confirma que no hay sobreajuste al conjunto de validación durante la optimización y que los modelos generalizan correctamente.

6. Selección y almacenaje del modelo baseline

Se selecciona la **Regresión Logística** como modelo baseline, en base a los tres criterios explicados abajo:

1. **F1 macro, la métrica considerada más importante por el desequilibrio de clases:** es esencialmente la misma para los dos, aunque muy ligeramente superior en el conjunto de test para la regresión logística (0.745 vs 0.744). Claramente, no es el factor determinante.

2. **Mayor recall sobre la clase de odio** (0.70 vs 0.64 en el conjunto de test): en una herramienta de apoyo a ONGs, los falsos negativos (discurso de odio no detectado) tienen un coste mayor que los falsos positivos. Cabe señalar que algunos falsos positivos pueden corresponder a lenguaje ofensivo o despectivo que no alcanza el umbral de discurso de odio, una distinción inherentemente ambigua que incluso se refleja en los datos de anotación, con etiquetas contradictorias. Por lo tanto, la mejor capacidad de recall de discurso de odio es un factor importante.

3. **Eficiencia computacional**: la regresión logística entrena en cuestión de segundos o pocos minutos frente a los ~40 minutos del Random Forest, lo que la hace más práctica para futuras iteraciones y reentrenamientos sobre nuevos datos.

Almacenaje: Tanto el modelo ajustado como el vectorizador TF-IDF se almacenan en el bucket *sinodio-models* de Cloud Storage bajo la ruta *phase1/*. Los resultados de ambos modelos se escriben en BigQuery como la tabla *baseline_ml_results* dentro del dataset *sinodio_results*, con un campo “selected” que identifica la Regresión Logística como el modelo baseline seleccionado.

H. Fine-tuning de XLM-RoBERTa

El fine-tuning de XLM-RoBERTa se llevó a cabo en notebooks/phase1/05_xlm_roberta.ipynb. El objetivo de esta fase era comparar, con la esperanza de superarlo, el rendimiento del baseline de ML tradicional, mediante el uso de un modelo transformer preentrenado.

Esto también respondía al requisito del proyecto de abordar el mismo problema con dos familias de técnicas distintas (Machine Learning tradicional y Deep Learning), y comparar sus resultados.

1. Infraestructura: VM en GCP con GPU

Dado el coste computacional de hacer fine-tuning de un modelo con 278 millones de parámetros (el portátil de desarrollo no cuenta con una GPU), el notebook se ejecutó en una máquina virtual en Google Cloud Platform. La configuración de la VM es la siguiente:

Parámetro	Valor
Tipo de instancia	n1-standard-4 (4 vCPUs, 15 GB RAM)
GPU	NVIDIA L4 (23,7 GB VRAM)
Región/Zona	europe-west4-a
Sistema operativo	Deep Learning VM with CUDA M130 (Ubuntu 22.04)
Aprovisionamiento	Standard

La GPU elegida inicialmente fue la NVIDIA T4, que es la opción más económica en GCP y suficiente para fine-tuning de XLM-RoBERTa. Sin embargo, la T4 no estaba disponible en ninguna zona de europe-west4 (la región donde ya se encuentran los contenedores o *buckets* del proyecto) en el momento de la creación de la VM. Por esta razón, se optó por la NVIDIA L4, una GPU de generación más reciente con mayor memoria (23,7 GB frente a 15,6 GB). Esto permitió explorar batch sizes más grandes (de hasta 64) durante la búsqueda de hiperparámetros con Optuna.

Se intentó inicialmente utilizar una máquina virtual (VM) con aprovisionamiento **Spot** para reducir costes (descuento del 60-70%), pero la VM fue interrumpida por GCP antes de completar el entrenamiento. Se intentó una segunda vez y pasó lo mismo, así que finalmente se optó por configurar una nueva VM con aprovisionamiento **Standard**, que tiene un coste de unos €0,50-0,60 por hora. Para evitar costes innecesarios, se configuró un límite automático de apagar la máquina al cabo de 4 horas.

Configuración del entorno

La imagen de Deep Learning VM incluye CUDA y los drivers de GPU preinstalados, pero no las librerías de Python necesarias. Se creó un fichero *requirements_vm.txt* específico para el entorno de la VM, con versiones compatibles con Python 3.10.12.

Las principales diferencias respecto al *requirements.txt* del portátil de desarrollo son:

- numpy==2.2.6 (frente a 2.4.3, no disponible para Python 3.10)
- pandas==2.3.3 (frente a 3.0.1)
- scikit-learn==1.7.2 (frente a 1.8.0)
- scipy==1.15.3 (frente a 1.17.1)

PyTorch requiere una instalación separada previa al resto de dependencias, ya que sus builds específicos de CUDA no están disponibles en el índice estándar de PyPI (*Python Package Index*):

```
pip3 install torch==2.6.0+cu124 --index-url https://download.pytorch.org/whl/cu124

pip3 install -r requirements_vm.txt
```

La versión torch==2.6.0+cu124 (compilada para CUDA 12.4) se eligió por compatibilidad con el entorno CUDA 12.8 de la VM, ya que PyTorch no disponía de builds para CUDA 12.8 en el momento del desarrollo, pero los builds de CUDA 12.4 son compatibles hacia adelante.

El acceso a Jupyter se configuró mediante una regla de firewall en GCP permitiendo el tráfico entrante en los puertos 8888-8889 exclusivamente desde la IP del ordenador portátil, garantizando que el servidor Jupyter no fuera accesible públicamente.

Limitaciones de permisos de la VM

Durante el desarrollo se identificaron dos limitaciones relacionadas con los permisos de la VM:

1. **Escritura en Cloud Storage**: el cliente Python google-cloud-storage no podía escribir en los buckets de GCS debido a que la VM fue creada sin los scopes de API necesarios. Como solución alternativa, la subida del modelo se realizó mediante gcloud storage cp después de la autentificación con gcloud auth login, que utiliza credenciales de usuario en lugar de las del service account de la VM.

2. **Escritura en BigQuery**: por la misma razón, los resultados no pudieron escribirse en BigQuery directamente desde la VM. Aquí, la solución encontrada fue de guardar los resultados en un fichero JSON local (*xlm_roberta_results.json*) y luego de subirlos a BigQuery directamente desde el ordenador portátil mediante un script independiente.

<u>Lección aprendida</u>: Ambas limitaciones se deben a que los scopes de API de una VM en GCP se definen en el momento de su creación y no pueden modificarse posteriormente. Por supuesto, en un entorno de producción, la VM se crearía con los scopes necesarios habilitados desde el inicio.

2. Tokenización

Se utilizó el tokenizador de XLM-RoBERTa con MAX_LENGTH=128 tokens. Esta decisión se basa en el análisis de la distribución de longitud de textos del EDA: la mediana de longitud es de aproximadamente 100 caracteres, y la tasa de truncación estimada a 128 tokens es de solo el 0,3%, lo que significa que muy pocos textos pierden información relevante por truncación. XLM-RoBERTa soporta hasta 512 tokens, pero usar 128 reduce significativamente el uso de

memoria y el tiempo de entrenamiento sin impacto apreciable en los datos. Si este modelo se extendiera a nuevos datasets con textos mucho más largos, obviamente se consideraría aumentar el parámetro MAX_LENGTH adecuadamente.

El tokenizador convierte cada texto en dos tensores:

- input_ids: la secuencia de tokens representada como índices enteros en el vocabulario de 250.002 subwords. Incluye el token especial [CLS] en la posición 0, que se utilizará como representación de la secuencia para la clasificación.
- attention_mask: secuencia de 1s y 0s que indica al modelo qué tokens son reales (1) y qué tokens son padding (0), para que el mecanismo de atención no procese los tokens de relleno.

3. Arquitectura del modelo

El modelo construído combina *XLM-RoBERTa base* como backbone, con una cabeza de clasificación personalizada:

- **XLM-RoBERTa base**: modelo transformer multilingüe preentrenado en 100 idiomas con 12 capas de atención y dimensión oculta de 768. Se eligió este modelo por su capacidad multilingüe, que permitirá extender el clasificador al francés en la Fase 2 sin cambios arquitectónicos.
- **Cabeza de clasificación personalizada** (768 → 256 → 2): en lugar de la capa lineal única estándar (768 → 2), se añade una capa intermedia con activación ReLU que aporta capacidad adicional para aprender representaciones específicas del discurso de odio antes de la clasificación final. Se aplica dropout (p=0,3) antes de cada capa lineal para regularizar el modelo.

El token [CLS] en la posición 0 del último estado oculto se utiliza como representación de la secuencia. Por las investigaciones hechas en este proyecto, este es el enfoque estándar para tareas de clasificación con modelos basados en RoBERTa, análogo al uso del mapa de características aplanado de una red convolucional (CNN) para clasificación de imágenes que en mi caso utilicé en el proyecto final de Deep Learning.

El modelo tiene **278.241.026 parámetros entrenables** y se ejecuta en GPU (cuda:0).

4. Búsqueda de hiperparámetros con Optuna

Se realiza una búsqueda de hiperparámetros en dos fases siguiendo una estrategia **coarse-to-fine**: primero una búsqueda amplia con pocos *epochs* para identificar la región prometedora del espacio de hiperparámetros, y después un entrenamiento completo más fino en la región prometedora para mejorar los parámetros encontrados.

Los resultados se presentan en la página siguiente.

Búsqueda coarse (7 *trials* completados en el tiempo límite de 2 horas):

Parámetro	Espacio de búsqueda
learning_rate	log-uniforme entre 1e-5 y 5e-5
batch_size	16, 32 o 64
warmup_ratio	uniforme entre 0,0 y 0,2

Cada *trial* entrenó durante 2 *epochs* y se evaluó por F1 macro en validación. Se utilizaron barras de progreso tqdm para monitorizar el entrenamiento en tiempo real, lo que permitió estimar la duración de cada trial y el tiempo total de la búsqueda.

Las barras tqdm, utilizadas también en algunos módulos del bootcamp, se implementaron como respuesta a una dificultad experimentada durante el entrenamiento del baseline de Random Forest, donde la ausencia de feedback de progreso y verbosidad hizo imposible saber por qué ciclo iba el modelo y así estimar el tiempo restante de entrenamiento. Con tiempos todavía más largos en Deep Learning, esta mejora era necesaria.

Los mejores parámetros encontrados fueron:

- learning_rate: 4,41e-05
- batch_size: 32
- warmup_ratio: 0,143
- **Mejor F1 macro (validación): 0,8097**

Los resultados se guardaron en *optuna_coarse_results.json* para garantizar su persistencia independientemente del estado del kernel de Jupyter. (Esto también fue una lección aprendida *the hard way*.)

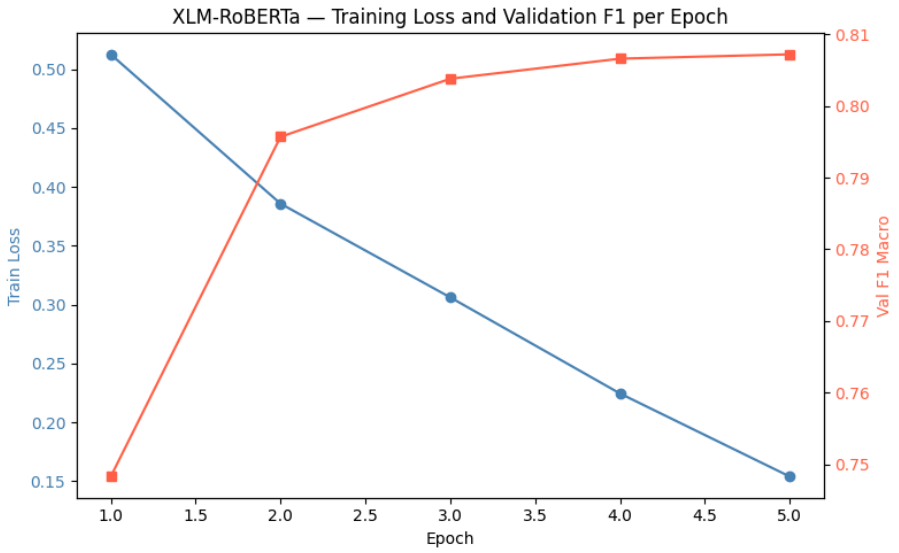
5. Fine-tuning completo

El modelo se entrenó durante 5 epochs con los mejores hiperparámetros encontrados, utilizando AdamW con weight_decay=0,01 y un scheduler de warmup lineal. Se implementó checkpointing para guardar el mejor modelo tras cada epoch que mejorara el F1 de validación, como medida de seguridad ante posibles interrupciones de la VM.

Evolución del entrenamiento:

Epoch	Train Loss	Val F1
1	0,5127	0,7483
2	0,3858	0,7957
3	0,3061	0,8038
4	0,2244	0,8066
5	0,1538	**0,8072**

El mejor modelo se guardó en el epoch 5. La función de pérdida de entrenamiento continúa descendiendo al final del entrenamiento y al mismo tiempo el F1 de validación se estabiliza, lo que sugiere que el modelo podría beneficiarse de 1 a 2 epochs adicionales, aunque el riesgo de sobreajuste aumentaría. Debido a los largos tiempos de entrenamiento, se decidió proceder con estos resultados.



La gráfica de entrenamiento mostrada arriba no se generó durante el proceso de fine-tuning debido a las restricciones operativas del entorno de ejecución: trabajar sobre una VM de GCP con coste por hora introduce presión para minimizar el tiempo de sesión, lo que llevó a priorizar guardar los resultados y liberar los recursos por encima de añadir visualizaciones adicionales. La gráfica se generó a posteriori a partir de los valores guardados en *config.json*, que se almacenaron precisamente como medida de precaución ante posibles interrupciones de la VM.

En el caso del baseline de ML tradicional (notebook 04), la evolución del entrenamiento no se visualizó porque Optuna gestiona el proceso de búsqueda internamente, es decir, no hay un bucle de entrenamiento por epochs explícito que permita registrar métricas epoch a epoch. Lo que sí se registró fue la evolución del F1 de validación por trial, que cumple una función análoga.

6. Evaluación y comparación con el baseline

Modelo	Split	Accuracy	Precisión	Recall	F1 Macro
Regresión Logística	val	0,776	0,738	0,749	0,743
Regresión Logística	test	0,775	0,739	0,754	0,745
XLM-RoBERTa	val	0,836	0,809	0,806	0,807
XLM-RoBERTa	test	0,840	0,814	0,807	**0,810**

XLM-RoBERTa mejora el F1 macro del baseline en **6,5 puntos porcentuales** (0,745 → 0,810), confirmando la hipótesis de que un modelo transformer multilingüe preentrenado es significativamente más capaz de capturar los patrones semánticos del discurso de odio que los

enfoques basados en TF-IDF. La mejora es consistente entre validación y test, lo que confirma que el modelo generaliza correctamente.

A nivel de clase, la mejora es especialmente pronunciada en la clase de odio, lo cual es particularmente interesante pensando en las posibles aplicaciones de uso de este modelo para ONGs:

- **Precisión en odio**: 0,62 (LR) → 0,75 (XLM-RoBERTa)
- **Recall en odio**: 0,70 (LR) → 0,72 (XLM-RoBERTa)
- **F1 en odio**: 0,65 (LR) → 0,74 (XLM-RoBERTa)

7. Análisis cualitativo del modelo

Como se explicó anteriormente, la distinción entre odio y no-odio no siempre es muy marcada, y en algunos casos los propios anotadores etiquetan un mismo documento de forma contradictoria. Esta subjetividad puede ser más pronunciada en los textos cortos tipo *tweets*, que precisamente constituyen la mayor parte de los datos utilizados. Así que los resultados obtenidos se consideran muy satisfactorios

Un análisis cualitativo del comportamiento del modelo con ejemplos reales, llevado a cabo una vez desplegada la API, se recoge en la sección I.5, junto con las limitaciones identificadas durante las pruebas.

8. Almacenamiento

El modelo fine-tuned, el tokenizador y la configuración de entrenamiento se almacenan en el bucket `sinodio-models` bajo la ruta `phase1/xlm_roberta/`. Los resultados de evaluación se escriben en BigQuery como la tabla *xlm_roberta_results* dentro del dataset `sinodio_results`.

I. Despliegue del modelo: FastAPI y Cloud Run

1. Infraestructura y enfoque

El despliegue del modelo se llevó a cabo siguiendo los conocimientos adquiridos en el módulo de *Despliegue de algoritmos* del bootcamp, adaptados a las particularidades del proyecto. El servicio de inferencia se construyó con FastAPI, el mismo framework utilizado en dicho módulo, y se desplegó en Google Cloud Run mediante un contenedor Docker.

La arquitectura de despliegue sigue el patrón estándar de MLOps: el modelo se carga desde Cloud Storage en el arranque del contenedor Docker, y el servicio muestra cuatro *endpoints* REST, (URL de la API a la que se puede hacer una petición de tipo *Representational State Transfer*, uno de los estilo arquitectónico más comunes para diseñar APIs web).

- GET / → información general del proyecto y enlaces a los demás endpoints
- GET /health → verificación de que el modelo está cargado y el servicio está operativo
- POST /predict → recibe un texto en español y devuelve la predicción binaria (odio / no odio) junto con las probabilidades de cada clase.
- POST/predict-batch →recibe una lista de hasta 50 textos y devuelve las predicciones individuales junto con un resumen estadístico (total, número de textos de odio, número de textos no-odio y porcentaje de discurso de odio).

Se optó por una imagen Docker con PyTorch en su versión CPU (torch==2.5.1+cpu) en lugar de la versión completa con CUDA, ya que la inferencia en CPU es suficiente para este caso de uso y reduce el tamaño de la imagen de ~2 GB a ~200 MB, lo que minimiza costes y tiempo de arranque en Cloud Run.

2. Dependencias del contenedor

Las dependencias del contenedor se gestionan en un fichero *requirements_api.txt* independiente, siguiendo la misma convención de nomenclatura utilizada en el proyecto para separar entornos (*requirements.txt* para el portátil de desarrollo, *requirements_vm.txt* para la VM de entrenamiento). Las versiones se han alineado con el entorno bd16-mlops del módulo de Despliegue:

- fastapi==0.115.6
- uvicorn==0.29.0
- transformers==4.47.0
- torch==2.5.1 (CPU)

3. Incidencias durante el despliegue

El proceso de despliegue requirió resolver una serie de incidencias relacionadas con permisos IAM y configuración de infraestructura, consistentes con las limitaciones de la organización de GCP identificadas ya durante el fine-tuning de XLM-RoBERTa (ver sección H.1):

- **Cloud Build API no habilitada**: al ejecutar gcloud builds submit por primera vez, el sistema solicitó habilitar la API cloudbuild.googleapis.com. Se habilitó mediante la interfaz de línea de comandos (CLI), progresivamente resolviendo los errores uno tras de otro.
- **Permisos insuficientes de la cuenta de Cloud Build**: el service account de Cloud Build (1064831214069@cloudbuild.gserviceaccount.com) no tenía acceso a Cloud Storage para leer el código fuente subido durante el build. Se resolvió asignando el rol roles/storage.admin mediante gcloud projects add-iam-policy-binding.
- **Permisos insuficientes de la cuenta de Compute**: el service account de Compute (1064831214069-compute@developer.gserviceaccount.com) presentaba la misma limitación. Se resolvió con el mismo procedimiento.

Estas incidencias son coherentes con el patrón observado a lo largo del proyecto: la política de seguridad de la organización de GCP restringe los permisos por defecto de los *service accounts*, requiriendo su asignación explícita. Como lección aprendida, en un proyecto futuro se configurarían estos permisos desde el inicio junto con el resto de la infraestructura en la fase de *Tools & Architecture*.

4. Interacción con la API

Una vez desplegado el servicio en Cloud Run, el modelo puede utilizarse de tres formas distintas, todas ellas basadas en los mismos endpoints REST:

a) Terminal con curl

Los endpoints se pueden llamar directamente desde la línea de comandos mediante curl. Por ejemplo, para clasificar un texto individual:

```
curl -X POST "https://sinodio-api-1064831214069.europe-west4.run.app/predict" \
  -H "Content-Type: application/json" \
  -d '{"text": "Ojalá que todos los inmigrantes se vayan de aquí"}'
```

b) Interfaz Swagger UI

FastAPI genera automáticamente una interfaz web interactiva accesible en la URL/docs del servicio. Esta interfaz, denominada Swagger UI, permite probar los endpoints directamente desde el navegador sin necesidad de escribir código. Para cada endpoint se muestran los parámetros de entrada, un ejemplo de cuerpo de petición editable, un botón "Execute" para lanzar la petición, y la respuesta del servidor con el código HTTP y el cuerpo JSON. Esta interfaz es equivalente a la utilizada en el módulo de *Despliegue de algoritmos* del bootcamp, y es especialmente útil para demostraciones y pruebas rápidas.

La interfaz Swagger UI del proyecto está disponible en:
https://sinodio-api-1064831214069.europe-west4.run.app/docs

Un ejemplo de utilización de esta interfaz se proporciona en la captura de pantalla siguiente, seguido de los resultados obtenidos:

SinOdio / SansHaine 1.0.0 OAS 3.1

/openapi.json

Binary hate speech classifier for Spanish (Phase 1). Fine-tuned XLM-RoBERTa on ~57k examples from Twitter and news comments.

default

GET / Root

GET /health Health

POST /predict Predict

POST /predict-batch Predict Batch

Classify a list of texts in one request.

Returns individual predictions for each text plus a summary: total count, hate speech count, non-hate count and hate speech percentage.

- **texts**: list of raw texts in Spanish (up to 50 texts per request)

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "texts": [
    "Ojalá que todos los inmigrantes se vayan de aquí",
    "Hoy ha sido un día tranquilo en el parque",
    "Las mujeres no deberían tener derecho a votar",
    "La diversidad cultural de mi barrio me permite practicar muchos idiomas pero al mismo tiempo
a veces me da miedo de que mi lengua cabe desapareciendo",
    "Estos refugiados son una plaga para nuestro país"
  ]
}
```


```
{
 "results": [
   {
     "text": "Ojalá que todos los inmigrantes se vayan de aquí",
     "text_clean": "Ojalá que todos los inmigrantes se vayan de aquí",
     "label": "hate_speech",
     "label_id": 1,
     "confidence": 0.9045,
     "prob_hate": 0.9045,
```

```
      "prob_no_hate": 0.0955
    },
    {
      "text": "Hoy ha sido un día tranquilo en el parque",
      "text_clean": "Hoy ha sido un día tranquilo en el parque",
      "label": "no_hate_speech",
      "label_id": 0,
      "confidence": 0.9982,
      "prob_hate": 0.0018,
      "prob_no_hate": 0.9982
    },
    {
      "text": "Las mujeres no deberían tener derecho a votar",
      "text_clean": "Las mujeres no deberían tener derecho a votar",
      "label": "no_hate_speech",
      "label_id": 0,
      "confidence": 0.9447,
      "prob_hate": 0.0553,
      "prob_no_hate": 0.9447
    },
    {
      "text": "La diversidad cultural de mi barrio me permite practicar muchos
idiomas pero al mismo tiempo a veces me da miedo de que mi lengua cabe
desapareciendo",
      "text_clean": "La diversidad cultural de mi barrio me permite practicar muchos
idiomas pero al mismo tiempo a veces me da miedo de que mi lengua cabe
desapareciendo",
      "label": "no_hate_speech",
      "label_id": 0,
      "confidence": 0.8908,
      "prob_hate": 0.1092,
      "prob_no_hate": 0.8908
    },
    {
      "text": "Estos refugiados son una plaga para nuestro país",
      "text_clean": "Estos refugiados son una plaga para nuestro país",
      "label": "hate_speech",
      "label_id": 1,
      "confidence": 0.5495,
      "prob_hate": 0.5495,
      "prob_no_hate": 0.4505
    }
  ],
  "total": 5,
  "hate_count": 2,
  "no_hate_count": 3,
  "hate_percent": 40
}
```

c) Notebook de Python

La forma más flexible de interactuar con la API es mediante la librería requests de Python, que permite integrar las predicciones del modelo en cualquier notebook o script del proyecto. El notebook *notebooks/phase1/06_api_demo.ipynb* demuestra el uso de los tres endpoints e incluye la visualización de los resultados en formato DataFrame. Esta modalidad es la más relevante para el uso real del modelo por parte de ONGs, ya que permite procesar colecciones de textos de forma programática y exportar los resultados para su análisis posterior.

La captura de pantalla siguiente muestra cómo se pueden obtener los resultados directamente en un DataFrame de Pandas, y de esta forma poder analizar los resultados más fácilmente.

	text	label	confidence	prob_hate	prob_no_hate
0	Ojalá que todos los inmigrantes se vayan de aq...	hate_speech	90.45%	90.45%	9.55%
1	Estos refugiados son una plaga para nuestro pa...	hate_speech	54.95%	54.95%	45.05%
2	Hoy ha sido un día tranquilo en el parque...	no_hate_speech	99.82%	0.18%	99.82%
3	La diversidad cultural de mi barrio me permite...	no_hate_speech	95.83%	4.17%	95.83%
4	Las mujeres no deberían tener derecho a votar...	no_hate_speech	94.47%	5.53%	94.47%
5	No entiendo por qué tenemos que acoger a tanta...	hate_speech	82.00%	82.00%	18.00%
6	@USER menuda panda de inútiles, todos a su paí...	hate_speech	99.22%	99.22%	0.78%

5. Análisis cualitativo y limitaciones identificadas durante el despliegue

Una vez desplegado el modelo, se realizó un análisis cualitativo mediante el endpoint /predict-batch de la API, clasificando un conjunto de ejemplos diseñados para explorar los límites del clasificador. Algunos de estos ejemplos están presentados en la subsección anterior de “Interacción con la API”.

Este análisis revela dos aspectos importantes que complementan las métricas cuantitativas reportadas en la subsección H. 6. “Evaluación y comparación con el baseline”.

En primer lugar, el **modelo clasifica correctamente y con alta confianza el discurso de odio explícito**: "Ojalá que todos los inmigrantes se vayan de aquí" se clasifica como odio con un 90,45% de confianza. Por contra, "Estos refugiados son una plaga para nuestro país" también se clasifica correctamente como discurso de odio, aunque con una confianza más baja (54,95%), lo que refleja que las **metáforas deshumanizadoras son más difíciles de detectar** que los insultos directos.

En segundo lugar, el modelo falla en la detección del **discurso de odio ideológico** dirigido a grupos minorizados o inferiorizados, en este caso las mujeres: "Las mujeres no deberían tener derecho a votar" se clasifica incorrectamente como no-odio con un 94,47% de confianza. Esta limitación es coherente con la composición del corpus de entrenamiento: el Superset, que representa el 52% de los datos, proviene mayoritariamente de Twitter, donde el discurso de odio tiende a expresarse de forma directa y explícita mediante insultos y lenguaje agresivo. El discurso de odio ideológico, que **niega derechos a grupos vulnerables** sin utilizar lenguaje explícitamente ofensivo, está probablemente infrarrepresentado en los datos de entrenamiento.

Estas observaciones tienen implicaciones directas para el trabajo futuro del proyecto. En primer lugar, justifican la construcción del corpus anotado manualmente, una etapa prevista para la Fase 3, con especial atención a ejemplos de discurso de odio implícito e ideológico dirigido a mujeres y minorías. Esto se vuelve particularmente importante en caso de usar el modelo con fines muy específicos, por ejemplo por parte de una ONG cuya misión sea defender los derechos de un subgrupo concreto de la población. En segundo lugar, sugieren que un clasificador multiclase por grupo objetivo (Fase 3) podría mejorar la detección al especializar el modelo en los patrones lingüísticos específicos de cada tipo de odio. Finalmente, el caso del comentario sobre refugiados ilustra que las metáforas deshumanizadoras ("una plaga") representan un reto específico que quizás se podría abordar mediante **técnicas de aumento de datos orientadas a este tipo de lenguaje figurado**.

J. Visualización de datos y resultados con dashboards de PowerBI

1. Dashboard de motivación del proyecto

a) Objetivo

El dashboard de motivación tiene como objetivo contextualizar el problema del discurso de odio en un marco más amplio, justificando la necesidad de herramientas automatizadas de detección como SinOdio/SansHaine. Para ello se utilizaron datos públicos de incidentes de odio registrados por la OSCE (Organización para la Seguridad y la Cooperación en Europa) a través de su plataforma ODIHR Hate Crime Reporting, que recopila datos de 47 estados participantes entre 2016 y 2024.

b) Fuente de datos

Los datos se descargaron en formato Excel desde https://hatecrime.osce.org y se importaron directamente en PowerBI Desktop. Durante la importación se detectó que PowerBI no reconocía automáticamente la primera fila como cabecera de columna y en su lugar añadía nombres genéricos (*Column1*, *Column2*, etc.). La solución fue acceder al Editor de Power Query (Inicio → Transformar datos) y aplicar la opción "Usar la primera fila como encabezado".

El dataset original contiene 32.994 incidentes de 54 países, con las siguientes columnas: fecha, país, motivación de odio, tipo de incidente, fuente y descripción. Para poder trabajar con los datos de forma eficiente en PowerBI, se añadieron dos columnas calculadas en Power Query:

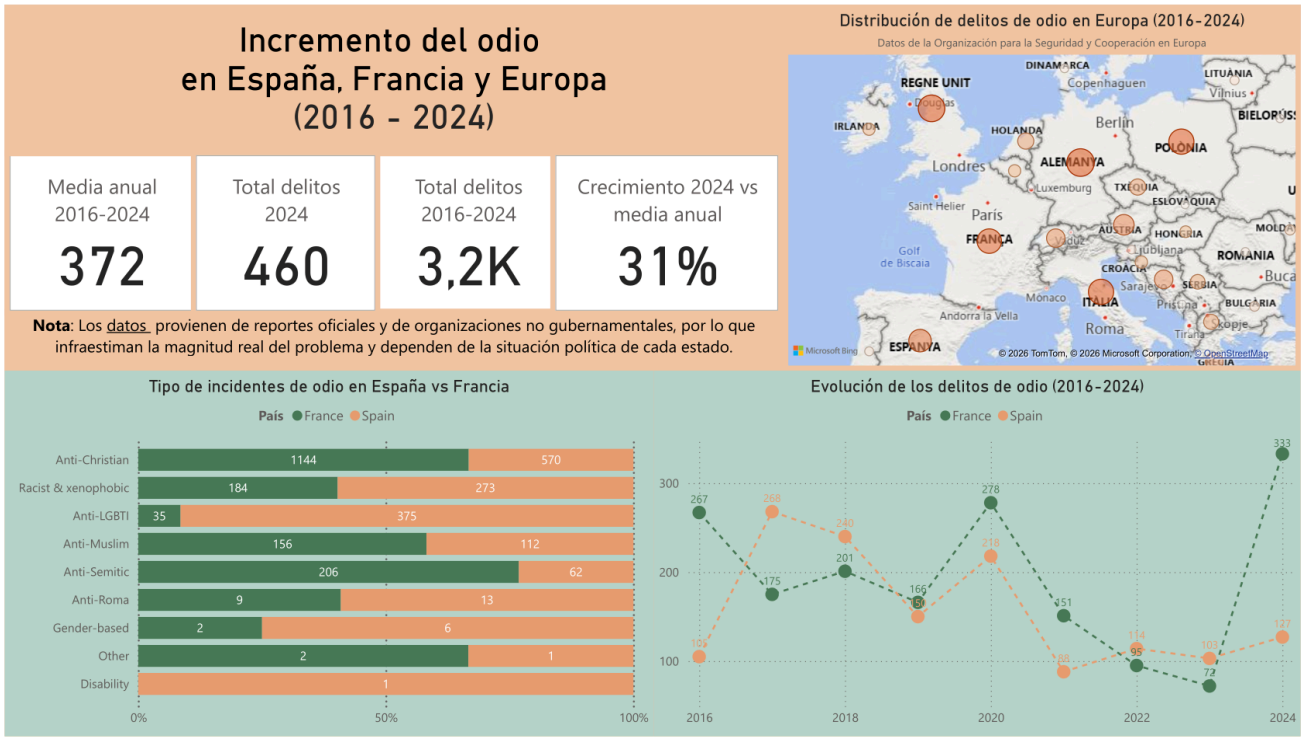
- **FullDate**: fecha completa en formato *Date*, generada añadiendo el sufijo "-01" al campo Date original (que contenía valores en formato "YYYY-MM"), mediante la fórmula `Date.FromText([Date] & "-01")`. Al asignar el tipo de dato *Date* a esta columna, PowerBI generó automáticamente una jerarquía temporal (año, trimestre, mes, día), habilitando las funciones nativas de inteligencia de tiempo.
- **Primary bias**: categoría simplificada de motivación de odio, obtenida mediante una columna personalizada con lógica condicional que asigna cada incidente a una de ocho categorías principales (Racist & Xenophobic, Anti-LGBTI, Anti-Semitic, Anti-Muslim, Gender-based, Anti-Roma, Disability, Anti-Christian) a partir del campo "Bias motivations", que en el dataset original contiene combinaciones de múltiples categorías.

Adicionalmente, se crearon dos tablas DAX calculadas para las visualizaciones centradas en España y Francia:

- **YearlyIncidents**: agrupa los incidentes de España y Francia por país, año y categoría de odio, utilizando `SUMMARIZE` con `FILTER` para restringir el conjunto a los dos países objetivo del proyecto.
- **BiasByCountry**: agrupa los incidentes de España y Francia por país y categoría de odio, para la visualización de la distribución por tipo.

c) Visualizaciones

El dashboard está compuesto por cuatro elementos visuales (además del título) que narran una situación enmarcada dentro del contexto europeo en el período 2016-2024, pero con foco específico en España y en Francia.



Las **tarjetas KPI** muestran cuatro indicadores de negocio: el promedio anual de incidentes en España y Francia combinados, el total de incidentes en 2024, el total acumulado 2016-2024, y el porcentaje de variación de 2024 respecto al promedio histórico del período 2016-2024. Este último KPI se calculó mediante una medida DAX con AVERAGEX sobre años para obtener el promedio de totales anuales (en lugar de AVERAGE sobre filas, que habría promediado valores a nivel de categoría produciendo un resultado incorrecto, una lección aprendida después de cometer el error).

El **mapa de burbujas** muestra la distribución geográfica de los incidentes registrados en todos los países de la OSCE entre 2016 y 2024, con el tamaño de cada burbuja proporcional al número total de incidentes. El mapa está centrado en Europa (con botones de zoom que permiten acercarse o alejarse) y utiliza una escala de color en tonos terracota, un color próximo al rojo para reforzar visualmente que los incidentes de odio son un fenómeno negativo. Cuánto más incidentes, más grande es el tamaño de la burbuja y más intenso es el color. Esta visualización pone de manifiesto que el problema está extendido por toda Europa, con concentraciones especialmente visibles en Europa occidental, y que existe un sesgo geográfico en los datos: los países con sistemas de registro más desarrollados presentan cifras más altas, lo que sugiere que las estadísticas oficiales subestiman la magnitud real del problema. Dentro de un mismo estado, los cambios de gobierno (y por tanto de las prioridades políticas) también pueden tener un impacto sobre la recopilación de datos.

El **gráfico de líneas** muestra la evolución anual de los incidentes en España y Francia entre 2016 y 2024, con líneas discontinuas y marcadores de puntos para reflejar que se trata de mediciones anuales discretas y no de una tendencia continua (se utilizó línea discontinua en lugar de continua porque la interpolación lineal entre puntos anuales sería un artefacto de la agregación temporal, y no un reflejo de la realidad). El gráfico revela patrones similares entre ambos países a partir de

2019, con una excepción notable: el pico de España en 2017 coincide con un mínimo local en Francia, y seguramente se puede relacionar con el referéndum de independencia de Cataluña, lo que ilustra cómo los eventos políticos pueden generar incrementos abruptos en los incidentes de odio registrados. Francia registra su máximo en 2024 (333 incidentes), el valor más alto de toda la serie para ambos países.

El **gráfico de barras apiladas** muestra la distribución de incidentes por categoría de odio en España y Francia. Destaca que en Francia las categorías dominantes de denuncias están relacionadas con delitos en base a la religión –con incidentes anticristianos (1144) mucho más frecuentes que los demás, antisemíticos (206) y antimusulmanes (156)– o con el racismo y la xenofobia (184), mientras que en España las categorías más relevantes son anticristianas (570), seguidas de anti-LGBTI (375) y racista-xenófobo (273). Esta distribución es relevante para el proyecto porque confirma que las categorías cubiertas por el corpus de entrenamiento de SinOdio/SansHaine (xenofobia, LGTBfobia y misoginia) están entre las más frecuentes en los dos países objetivo.

d) Vías de mejora

El dashboard en su versión actual no incluye slicers interactivos de año o país, que habrían permitido al usuario filtrar todas las visualizaciones simultáneamente. Esta funcionalidad sería fácil de implementar porque la jerarquía temporal ya está activada, pero tampoco parecía una prioridad, ya que el dashboard en su versión actual cumple su propósito de justificar la necesidad del proyecto, en un momento en que los discursos y delitos de odio se están incrementando en muchas partes de Europa.

2. Dashboard de resultados del modelo

a) Conexión a BigQuery

La visualización de los resultados del modelo se realizó mediante dos dashboards interactivos en PowerBI Desktop, conectados directamente al dataset *sinodio_results* de BigQuery. La conexión se configuró mediante una cuenta organizativa de Google (OAuth), dado que la política de seguridad de la organización de GCP deshabilita la creación de claves de cuenta de servicio, como ya se documentó en la sección B.1.

Durante la configuración se identificó una incidencia con el nuevo conector de BigQuery de PowerBI (ADBC Implementation 2.0): este conector no gestiona correctamente los datasets alojados fuera de la región US, devolviendo el error *"Dataset was not found in location US"* aunque el dataset existe y es accesible. La solución fue deshabilitar la nueva implementación desde *Archivo → Opciones → Preview Features → BigQuery connector*, revirtiendo al conector antiguo, que sí respeta la región del dataset.

Al conectar, PowerBI ofrece dos modos de importación: **Import** (copia los datos localmente en el modelo de PowerBI) y **DirectQuery** (consulta BigQuery en tiempo real en cada interacción). Se optó por **Import** por dos razones: las tablas de resultados son pequeñas y estáticas (no cambian durante el desarrollo), y DirectQuery genera consultas a BigQuery con cada interacción del usuario, lo que tiene implicaciones de coste en datasets grandes o dashboards con mucho tráfico. En un

escenario de producción con datos que se actualizan frecuentemente (por ejemplo, predicciones en tiempo real sobre nuevos textos) DirectQuery sería la opción adecuada.

b) Modelado de datos con DAX

Los resultados del modelo están almacenados en dos tablas separadas en BigQuery: *baseline_ml_results* (con resultados de Regresión Logística y Random Forest) y *xlm_roberta_results* (con resultados de XLM-RoBERTa). Para poder comparar ambos modelos en las mismas visualizaciones, fue necesario combinar estas tablas en una estructura unificada mediante tablas calculadas en DAX (*Data Analysis Expressions*), el lenguaje de fórmulas nativo de PowerBI.

Se crearon tres tablas calculadas:

- **ModelResults**: combina los resultados en test de ambas tablas en una única tabla de dos filas (una por modelo) mediante `UNION` y `SELECTCOLUMNS`, con `FILTER` para seleccionar únicamente el split de test. Esta tabla se utilizó como base para las tarjetas KPI (*Key Performance Indicator*).

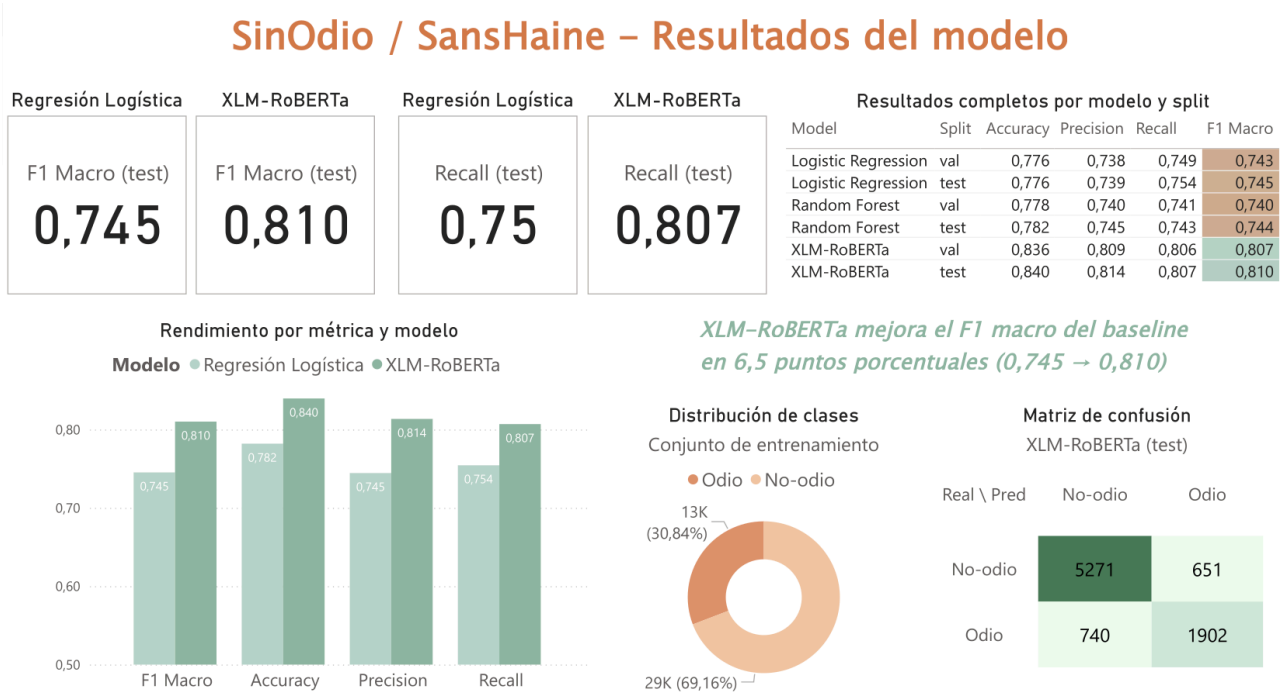
- **ModelResultsLong**: transforma la estructura ancha (una columna por métrica) en una estructura larga (una fila por combinación modelo-métrica), añadiendo una columna *SortOrder* para controlar el orden de las métricas en el eje X del gráfico de barras agrupadas. Esta transformación, análoga al concepto de *unpivot* o *melt* en pandas, es necesaria para que PowerBI pueda representar múltiples métricas como categorías del eje X con los modelos como leyenda.

- **AllResults**: combina todas las filas de ambas tablas originales (validación y test, todos los modelos) para la tabla de resultados completos del dashboard.

- **ConfusionMatrix**: tabla estática con los valores de la matriz de confusión de XLM-RoBERTa en el conjunto de test, calculados a partir de los valores de precisión, recall y soporte reportados en el notebook de evaluación.

El uso de tablas DAX calculadas permitió mantener las tablas originales de BigQuery sin modificaciones, aplicando todas las transformaciones necesarias en la capa de modelado de PowerBI. En el módulo de Exploración y Visualización de Datos ya aprendimos que este tipo de separación entre los datos y el modelo de presentación es una buena práctica en el desarrollo de dashboards, ya que facilita el mantenimiento y la actualización de los datos sin necesidad de reconstruir las visualizaciones.

c) Dashboard de Resultados



El dashboard de resultados mostrado arriba está estructurado en cinco elementos visuales que, en conjunto, ofrecen una visión completa del rendimiento del sistema, desde una perspectiva tanto técnica como orientada al negocio.

Indicadores KPI

Los cuatro indicadores KPI en la parte superior izquierda del dashboard sintetizan de un vistazo las dos métricas más relevantes, *F1 macro* y *recall*, para evaluar la utilidad del sistema en su contexto de aplicación real. La elección de estos indicadores no es arbitraria: responde a una lógica de negocio concreta orientada al caso de uso del proyecto, que es servir de herramienta de apoyo a ONGs en la detección de discurso de odio.

Se muestran el *F1 macro* de XLM-RoBERTa (0,810) y de la Regresión Logística (0,745) como métrica principal de comparación entre modelos, ya que el *F1 macro* pondera por igual el rendimiento sobre ambas clases independientemente de su frecuencia, lo que lo convierte en la métrica más adecuada para un corpus desequilibrado. Se muestra además el *recall* de XLM-RoBERTa (0,807) y de la Regresión Logística (0,754) como segundo KPI de negocio: en el contexto de detección de discurso de odio para ONGs, el recall, que corresponde a la proporción de casos de odio reales que el modelo ha detectado correctamente, tiene una importancia especial, ya que un falso negativo (discurso de odio que no se detectó) tiene un coste mayor que un falso positivo (texto inofensivo clasificado erróneamente como odio). La mejora de XLM-RoBERTa en *recall* (+5,3 puntos porcentuales) es por tanto tan relevante como su mejora en *F1 macro*.

Gráfico de barras agrupadas: rendimiento por métrica y modelo

El gráfico de barras agrupadas muestra las cuatro métricas de evaluación (*F1 macro*, *accuracy*, *precision* y *recall*) para ambos modelos en el conjunto de test, permitiendo una comparación visual directa. XLM-RoBERTa supera a la Regresión Logística en todas las métricas, confirmando que la inversión computacional del fine-tuning de un modelo transformer está justificada.

La *accuracy* (0,840 vs 0,775) mide la proporción global de predicciones correctas sobre todas las clases, pero es una métrica que puede resultar engañosa en presencia de desequilibrio de clases: un clasificador que predijese siempre "no odio" obtendría una accuracy del 69% sin detectar ningún caso de odio. La *precisión* (0,814 vs 0,739) mide qué proporción de los textos clasificados como odio son realmente odio, es decir, la fiabilidad de las alertas generadas. En el contexto de la aplicación real para ONGs de este proyecto, la *precisión* es menos importante que el *recall* (0,807 vs 0,754), que mide qué proporción del discurso de odio real es detectada por el modelo. El *F1 macro* (0,810 vs 0,745) es la media armónica de precisión y recall calculada por clase e igualmente ponderada, y constituye el mejor resumen global del rendimiento del clasificador en este contexto.

Distribución de clases del corpus

El gráfico de donut muestra la distribución de clases en el conjunto de entrenamiento: 69,16% de textos sin discurso de odio y 30,84% con discurso de odio. Este desequilibrio es realista —en datos reales de redes sociales el discurso de odio es genuinamente minoritario— y tiene implicaciones directas para el modelado: justifica el uso de *class_weight='balanced'* en los modelos de ML tradicional y el seguimiento del F1 macro como métrica principal en lugar de la accuracy. Cabe destacar que la división del corpus en los tres conjuntos (entrenamiento, validación y test) se realizó con estratificación por clase (*stratify=label*), lo que garantiza que esta distribución 69%/31% se mantiene de forma consistente en los tres splits y que los resultados de evaluación no están sesgados por una distribución de clases distinta en el conjunto de test. La visualización de esta distribución en el dashboard contextualiza el resto de métricas para cualquier usuario no técnico, como podría ser el personal de una ONG que utilice el sistema.

Matriz de confusión

La matriz de confusión es la visualización más informativa para entender el comportamiento detallado del clasificador más allá de las métricas agregadas. Muestra el desglose completo de las predicciones de XLM-RoBERTa sobre los 8.564 ejemplos del conjunto de test, organizado en cuatro categorías:

Los **5.271 verdaderos negativos** (Real: No-odio, Pred: No-odio) son textos inofensivos correctamente identificados como tales — el caso más frecuente, consistente con la clase mayoritaria del corpus.

Los **1.902 verdaderos positivos** (Real: Odio, Pred: Odio) son casos de discurso de odio correctamente detectados por el modelo — el resultado más valioso desde la perspectiva de la ONG, ya que representan las alertas correctas que el sistema generaría.

Los **651 falsos positivos** (Real: No-odio, Pred: Odio) son textos inofensivos clasificados erróneamente como odio. En el contexto de uso del sistema, estos generarían falsas alertas que requerirían revisión humana — un coste asumible si el volumen es manejable.

Los **740 falsos negativos** (Real: Odio, Pred: No-odio) son casos de discurso de odio que el modelo no detectó. Estos representan el error más costoso desde la perspectiva del caso de uso: contenido dañino que pasaría desapercibido sin revisión humana adicional. El hecho de que XLM-RoBERTa cometa menos falsos negativos que la Regresión Logística (740 vs aproximadamente 793) es una de las razones clave para su selección como modelo final.

El formato de tabla con intensidad de color facilita la lectura intuitiva de la matriz: las celdas de la diagonal principal (verdaderos positivos y negativos) aparecen en tonos más oscuros, destacando visualmente los aciertos del modelo frente a los errores.

Tabla de resultados completos

La tabla de resultados completos recoge todas las métricas de evaluación para los tres modelos entrenados (Regresión Logística, Random Forest y XLM-RoBERTa) tanto en validación como en test, con el F1 macro destacado mediante formato condicional. Esta tabla permite verificar que no hay sobreajuste al conjunto de validación —los resultados son consistentes entre val y test en todos los modelos— y ofrece una referencia completa para cualquier consulta técnica durante la presentación.

K. Conclusiones, lecciones aprendidas y trabajo futuro

1. Validación de las suposiciones iniciales

Al inicio del proyecto se formularon una serie de suposiciones que el desarrollo posterior ha podido contrastar. Las principales se analizan a continuación.

Suposiciones confirmadas:

La hipótesis central del proyecto, que un modelo transformer multilingüe preentrenado superaría significativamente a los enfoques clásicos de ML para la detección de discurso de odio en español, queda confirmada con claridad: XLM-RoBERTa alcanza un F1 macro de 0,810 en el conjunto de test, frente a 0,745 de la Regresión Logística con TF-IDF, una mejora de 6,5 puntos porcentuales consistente tanto en validación como en test. La mejora es especialmente pronunciada en la clase minoritaria de odio (F1 en odio: 0,65 → 0,74), que es la más relevante para el caso de uso objetivo del proyecto por parte de ONGs.

La decisión de combinar tres datasets heterogéneos -el Superset (Twitter, español latinoamericano), DETOXIS (comentarios de noticias, xenofobia) y HAHA (humor ofensivo)- también resultó acertada. El análisis de vocabulario por clase del EDA reveló perfiles lingüísticos muy distintos entre los tres corpus: un modelo entrenado sobre uno solo de ellos probablemente no generalizaría bien a los otros dos registros. La combinación de los tres obliga al modelo a aprender representaciones más robustas del discurso de odio más allá de un único registro o grupo-objetivo.

La elección de la Regresión Logística frente a Random Forest como modelo baseline también quedó validada: Random Forest mostró un recall claramente inferior en la clase de odio (0,64 vs 0,70 en el conjunto de test), confirmando su conocida limitación ante datos dispersos de alta dimensionalidad como los vectores TF-IDF.

Suposiciones no confirmadas:

La hipótesis de que el comportamiento de mención masiva ("@USER") podría ser un indicador discriminante de discurso de odio no se confirmó en los datos: la media y la mediana de menciones "@USER" son similares en ambas clases (0,97 vs 0,89), lo que sugiere que los patrones de acoso están codificados en el contenido semántico del texto más que en sus características estructurales. Este hallazgo del EDA reforzó la elección de XLM-RoBERTa, cuyo mecanismo de atención captura relaciones contextuales que las características estructurales -incluso con trigramas- no pueden detectar.

La suposición de que las máquinas virtuales con aprovisionamiento Spot de GCP serían suficientemente estables para el entrenamiento de modelos con tiempos de varias horas resultó incorrecta: la VM fue interrumpida por GCP en dos ocasiones antes de completar el fine-tuning, lo que obligó a migrar a aprovisionamiento Standard con un coste aproximado de 0,50–0,60 €/hora. Esta experiencia subraya que el aprovisionamiento Spot es adecuado para tareas tolerantes a interrupciones y de corta duración, pero no para entrenamientos largos de deep learning sin checkpointing robusto.

2. Valoración de las métricas seleccionadas

La métrica principal de evaluación elegida fue el F1 macro, que calcula la media no ponderada del F1 de cada clase. Esta elección fue correcta para el caso de uso concreto del proyecto por tres razones. En primer lugar, el corpus presenta un desequilibrio de clases moderado (69% no odio / 31% odio) que hace que la exactitud (accuracy) sea una métrica engañosa: un clasificador que predijese siempre la clase mayoritaria obtendría una accuracy del 69% sin detectar ningún caso de odio. En segundo lugar, el contexto de aplicación (herramienta de apoyo a ONGs) hace que los falsos negativos (discurso de odio no detectado) tengan un coste mayor que los falsos positivos, lo que justifica prestar especial atención al recall de la clase de odio. En tercer lugar, la misma métrica se utilizó en el módulo de NLP del bootcamp para tareas de clasificación de reviews con desequilibrio de clases, lo que facilita la comparabilidad con el estado del arte en la literatura especializada.

Como complemento, se reportaron también la accuracy, la precisión y el recall por clase, que ofrecen una imagen más completa del comportamiento del modelo. El análisis cualitativo de predicciones concretas (sección I.5) añade una dimensión interpretativa que las métricas cuantitativas por sí solas no capturan.

3. Valoración de la arquitectura elegida

La arquitectura del proyecto ha sido adecuada para los objetivos de la Fase 1:

- GCP como plataforma de nube
- Cloud Storage para almacenamiento de datos y modelos
- BigQuery como repositorio central de métricas
- Cloud Run para el despliegue del servicio de inferencia

Además ha resultado práctica para trabajar sin problemas con dos particiones distintas del portátil de desarrollo: Linux para la mayoría del trabajo y Windows para la parte de visualización con PowerBI.

El modelo de facturación por petición de Cloud Run resultó útil y acertado para un despliegue en fase inicial (sin coste cuando no hay tráfico). Además, su soporte nativo de contenedores Docker y su capacidad de escalar automáticamente lo hacen idóneo para un prototipo de investigación con tráfico variable. La elección de una imagen Docker con PyTorch en versión CPU (en lugar de la versión CUDA completa) redujo el tamaño de la imagen de ~2 GB a ~200 MB, minimizando costes y tiempos de arranque en frío.

La principal limitación identificada en la arquitectura es la política de permisos de la organización de GCP, que deshabilita la creación de claves de cuenta de servicio y restringe los permisos por defecto de los service accounts. Esto generó problemas recurrentes a lo largo del proyecto (escritura en Cloud Storage desde la VM, escritura en BigQuery, permisos de Cloud Build). En un proyecto futuro, se configuraría toda la infraestructura IAM (incluyendo los scopes de las VMs y los roles de los service accounts de Cloud Build y Compute) desde la fase inicial de *Tools & Architecture*, antes de comenzar el desarrollo.

4. Lecciones aprendidas

El desarrollo de este proyecto ha generado un conjunto de lecciones aprendidas de valor práctico para las extensiones de este proyecto y para proyectos futuros.

Infraestructura y configuración:

Los permisos IAM en GCP deben configurarse de forma exhaustiva al inicio del proyecto, incluyendo los scopes de las VMs y los roles de todos los service accounts que participarán en el pipeline. La ausencia de esta configuración inicial generó trabajo de resolución de incidencias recurrente a lo largo del desarrollo. Del mismo modo, los scopes de API de una VM no pueden modificarse una vez creada, por lo que deben definirse correctamente en el momento de su aprovisionamiento.

El aprovisionamiento Spot en GCP es adecuado para tareas cortas y tolerantes a interrupciones, pero no para entrenamientos largos de aprendizaje profundo (*deep learning*) como este proyecto. Para estos casos, el coste adicional del aprovisionamiento Standard (aproximadamente el doble) queda justificado por la estabilidad que proporciona.

Gestión de credenciales:

Las credenciales deben gestionarse mediante variables de entorno o archivos *.env* excluídos del control de versiones desde el primer *commit* del repositorio, no como medida correctiva tras un incidente. La exposición accidental del token de HuggingFace en el código del notebook, aunque sin consecuencias graves, ilustra la importancia de establecer estas prácticas desde el inicio.

Reutilización y mantenibilidad del código:

El pipeline de ML tradicional gestionó el vectorizador TF-IDF y el clasificador como objetos independientes. En retrospectiva, encapsularlos en un Pipeline de scikit-learn habría simplificado el código, prevenido fugas de datos de forma nativa, y facilitado el reentrenamiento sobre nuevos datos –algo especialmente relevante dado que las Fases 2 y 3 del proyecto implican extender el clasificador a nuevos idiomas y categorías. Los Pipelines de scikit-learn proporcionan además una visualización nativa del flujo completo de preprocesamiento y modelado, útil tanto para documentación como para depuración. Esta mejora se incorporará en la siguiente iteración del baseline.

Robustez del proceso de entrenamiento:

En entornos de VM con coste por hora, la presión para minimizar el tiempo de sesión puede llevar a omitir medidas de seguridad básicas. Este proyecto ha demostrado el valor de implementar checkpointing por epoch, guardar los resultados intermedios en ficheros JSON independientes del kernel de Jupyter, y utilizar barras de progreso (*tqdm*) para monitorizar el avance y estimar tiempos. Estas medidas tienen un coste de implementación bajo, permiten una mejor gestión del tiempo y evitan pérdidas de trabajo significativas ante interrupciones inesperadas.

Análisis cualitativo como complemento imprescindible:

Las métricas cuantitativas como el F1 macro no capturan todos los aspectos relevantes del comportamiento de un clasificador. El análisis cualitativo de predicciones concretas (en particular la clasificación errónea de "Las mujeres no deberían tener derecho a votar" como no-odio) reveló limitaciones del modelo que no son visibles en las métricas agregadas y que tienen implicaciones directas para el diseño de fases futuras del proyecto, como la aumentación de datos.

Import vs DirectQuery en PowerBI:

Durante la conexión de PowerBI a BigQuery surgió la decisión de elegir entre dos modos de importación: **Import** y **DirectQuery**. El modo Import copia los datos localmente en el modelo de PowerBI en el momento de la conexión, mientras que DirectQuery consulta la fuente de datos en tiempo real cada vez que el usuario interactúa con el dashboard. Para este proyecto se eligió Import por dos razones: las tablas de resultados son pequeñas y estáticas (los resultados del modelo no cambian durante el desarrollo), y DirectQuery genera consultas a BigQuery con cada interacción, lo que tiene implicaciones de coste en datasets grandes o dashboards con mucho tráfico. En un escenario de producción con datos que se actualizan frecuentemente (por ejemplo, predicciones en tiempo real sobre nuevos textos entrantes), DirectQuery sería la opción adecuada, ya que garantizaría que el dashboard reflejara siempre el estado más reciente de los datos sin necesidad de actualizar manualmente el modelo.

GitHub Projects como herramienta de gestión:

La gestión del proyecto mediante GitHub Projects con un tablero de tres columnas (*To Do*, *In Progress*, *Done*) y seis milestones demostró ser una herramienta de gran valor práctico, sin la cual no hubiera podido acabar este proyecto. En un proyecto complejo con múltiples fases interdependientes, el tablero permitió mantener el foco en la tarea en curso sin perder de vista las dependencias entre tareas: por ejemplo, la decisión de elegir XLM-RoBERTa como backbone arquitectónico en la Fase 1 tenía implicaciones directas sobre la Fase 2 (extensión al francés) y sobre el despliegue, y tener estas dependencias documentadas como issues etiquetadas facilitó tomar decisiones informadas en cada momento. Como lección aprendida, se recomienda invertir tiempo al inicio del proyecto en definir los milestones y las issues con suficiente detalle, ya que este esfuerzo inicial se amortiza rápidamente a lo largo del desarrollo.

5. Trabajo futuro

El proyecto está diseñado desde su inicio como un sistema incremental de cuatro fases. La Fase 1, documentada en esta memoria, constituye un entregable completo y funcional: un clasificador binario de discurso de odio en español con F1 macro de 0,810, desplegado como API REST en producción. Las fases siguientes extienden sus capacidades de forma natural:

A corto plazo (inmediatamente después del bootcamp):

Se prevé publicar el modelo fine-tuned en HuggingFace Hub y desplegar una interfaz de demostración pública en HuggingFace Spaces mediante Gradio, lo que permitirá que cualquier persona interesada (investigadores, ONGs, periodistas) pueda probar el clasificador de forma interactiva sin necesidad de acceder a la API REST. Esta interfaz también serviría como canal de recogida de ejemplos para ampliar el corpus de entrenamiento.

Fase 2 - Extensión al francés (y potencialmente a otros idiomas):

La elección de XLM-RoBERTa como backbone facilita esta extensión: el modelo ya fue preentrenado en 100 idiomas, incluyendo el francés, por lo que el fine-tuning adicional sobre los datasets CONAN y OLID-FR puede realizarse sin cambios arquitectónicos. Esta fase cubre el ámbito geográfico del proyecto (España y Francia) y amplía el impacto potencial del sistema. Más allá, también se podrían incluir otros idiomas menos desarrollados actualmente en los LLMs.

Fase 3 - Clasificador multiclase:

El análisis cualitativo de la Fase 1 revela que el modelo comete errores sistemáticos en la detección de discurso de odio implícito e ideológico dirigido a determinados grupos (en particular, mujeres). Un clasificador multiclase por grupo objetivo –incluyendo misoginia, xenofobia, LGTB-fobia, antisemitismo, catalanofobia, aporofobia, y otros– permitiría especializar el modelo en los patrones lingüísticos específicos de cada tipo de odio y mejorar la detección de formas de odio que actualmente se escapan al clasificador binario.

Fase 4 - Recuperación de contranarraciones:

La fase final del proyecto converge las ramas español y francés con la capacidad de recuperar contranarraciones efectivas previamente validadas por ONGs (corpus CONAN). Esta funcionalidad transforma el sistema de una herramienta de detección pasiva a una herramienta de respuesta activa, alineada con el objetivo original del proyecto de servir de apoyo a las organizaciones que trabajan contra el discurso de odio en línea.